

DEPARTEMENT MATHÉMATIQUE
FILIERE: LICENCE EN MATHÉMATIQUE ET INFORMATIQUE

PROJET DE FIN D'ANNÉE

Automatisation de l'Analyse des Ventes E-commerce

Développement d'une solution de traitement de données sous Python

Présenté par :

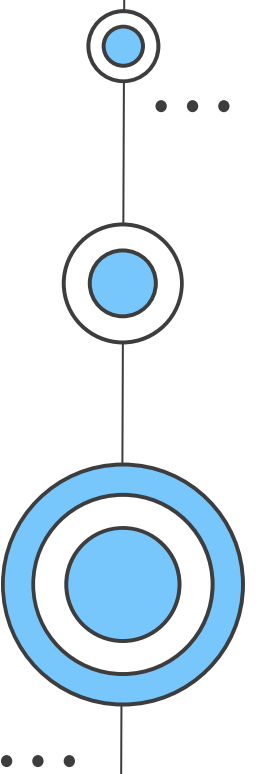
Takwa Labidi
Hamza Mokrani
Yassine Zouaoui



Encadrante:
Mme imene

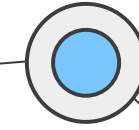
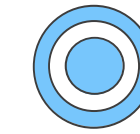
Matière : Logiciel

Date : Avril 2026

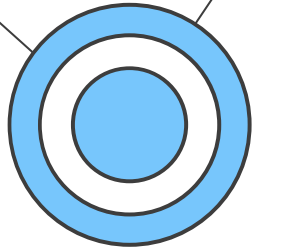


Plan du projet

...



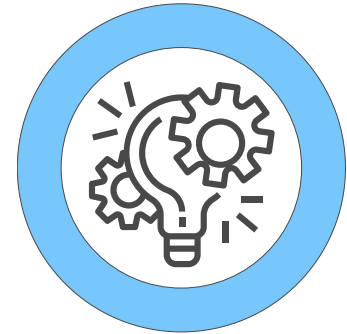
...



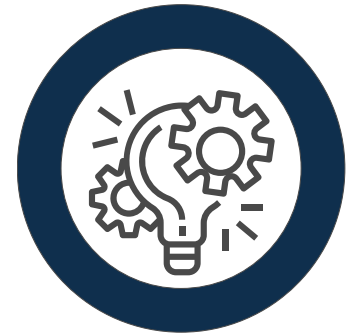
...



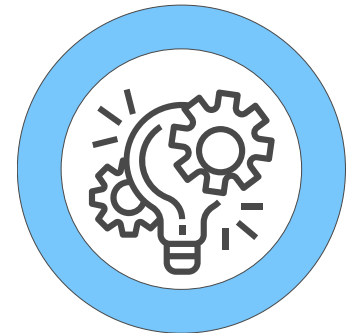
1. Analyse du Besoin et Conception



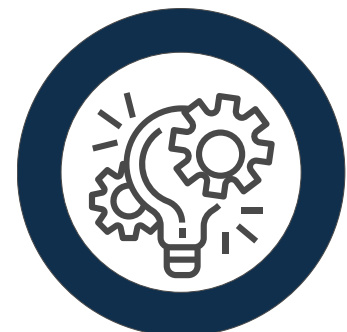
2. Mise en place de l'Environnement de Travail



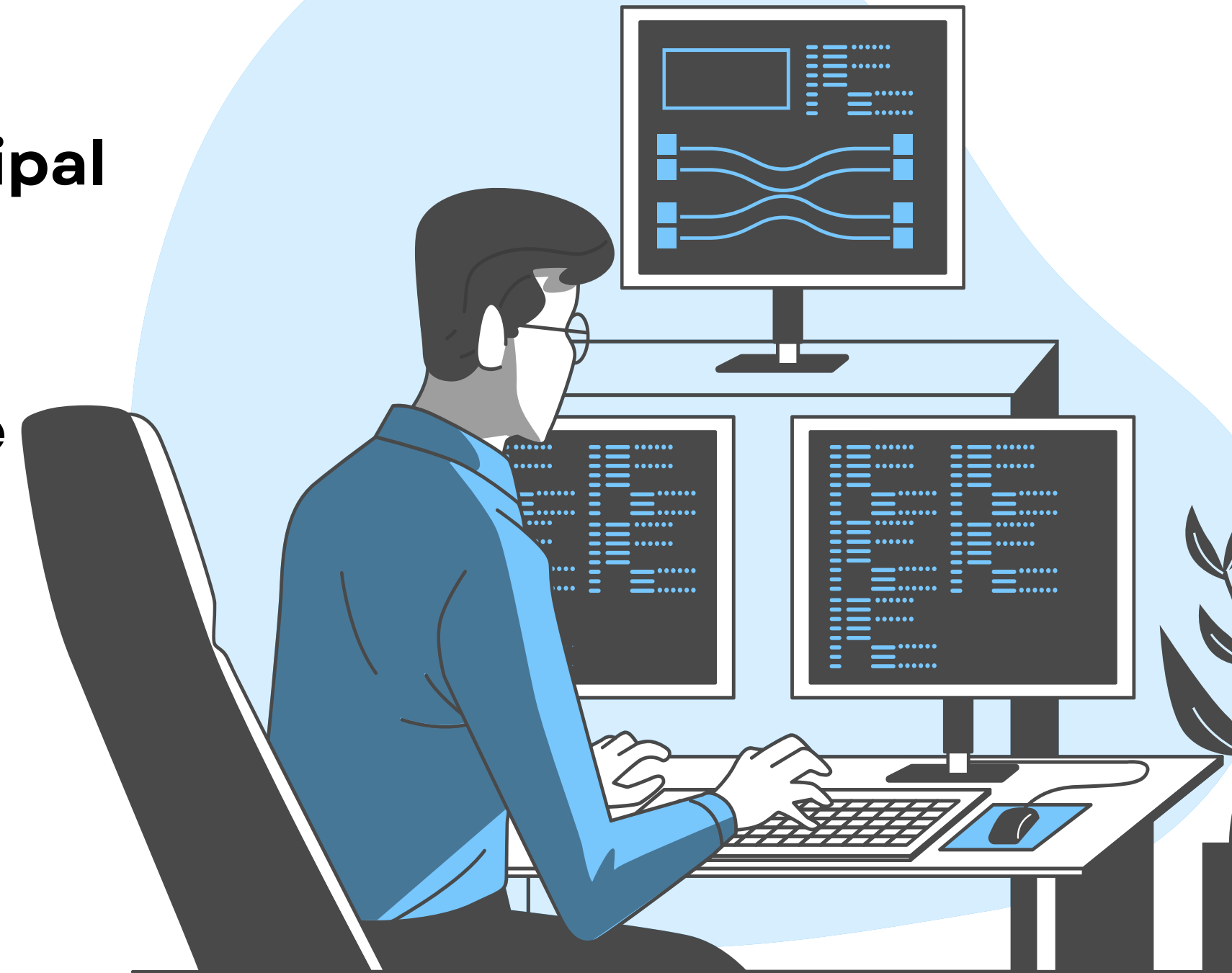
**3. Développement du Script Principal
(main.py)**



**4. Tests de Robustesse et "Lecture
Dynamique"**



5. Documentation et Versioning



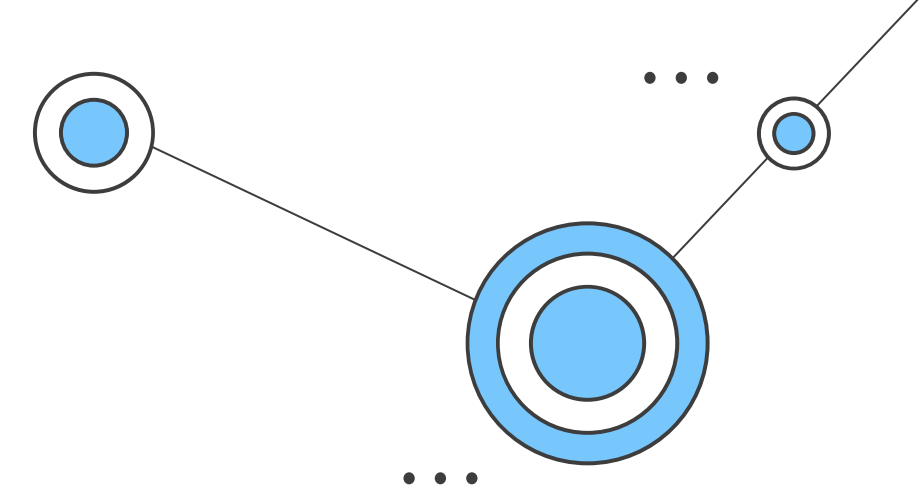
Plan du projet



1. Analyse du Besoin et Conception



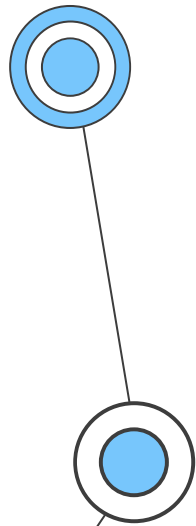
Analyse du Besoin et Conception



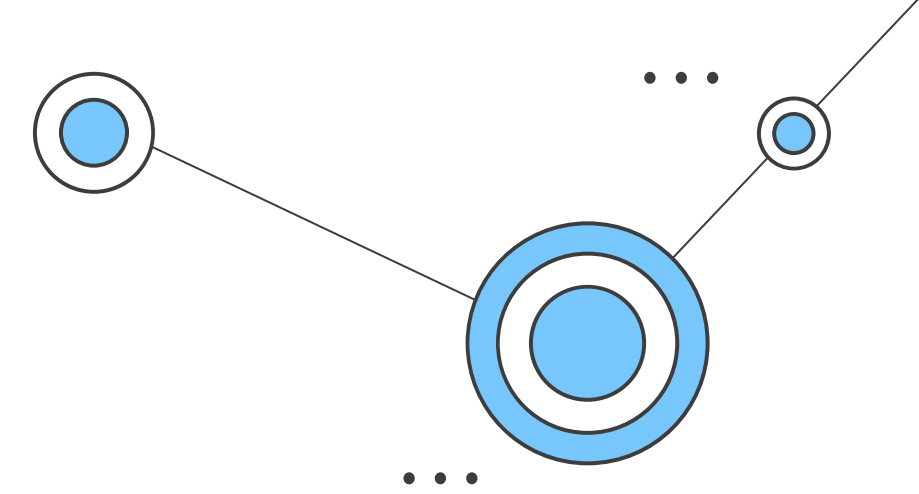
1. Objectifs du Projet

L'enjeu principal de ce travail réside dans le passage **d'une gestion manuelle** sur tableur à une solution entièrement **automatisée**. Face à un volume de données croissant, l'objectif est de concevoir un script capable de traiter un fichier brut pour en extraire des indicateurs financiers précis.

Cela inclut le calcul automatisé du **Chiffre d'Affaires brut**, l'application **de remises commerciales** et la détermination de la **TVA** à 20 %, tout en identifiant le produit le plus rentable de l'entreprise.

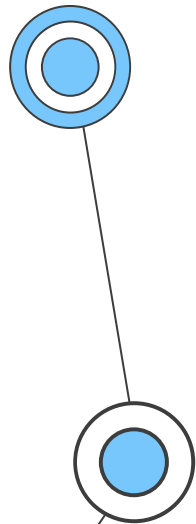


2. Choix Technologiques



Pour répondre à ces besoins d'analyse, le langage **Python** a été retenu pour sa flexibilité et sa puissance de calcul. L'implémentation s'appuie principalement sur **la bibliothèque Pandas**, un standard de l'industrie pour la manipulation de données structurées.

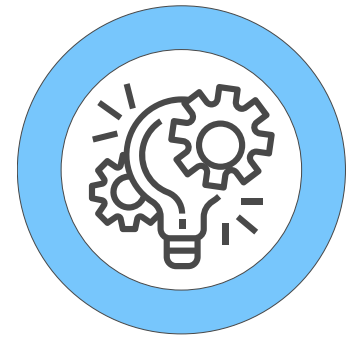
Ce choix permet de gérer efficacement **le format CSV**, d'effectuer des opérations vectorisées sur les colonnes (Prix, Quantité, Remise) et de générer un fichier de sortie complet (resultats_final.csv) de manière rapide et fiable.



Plan du projet



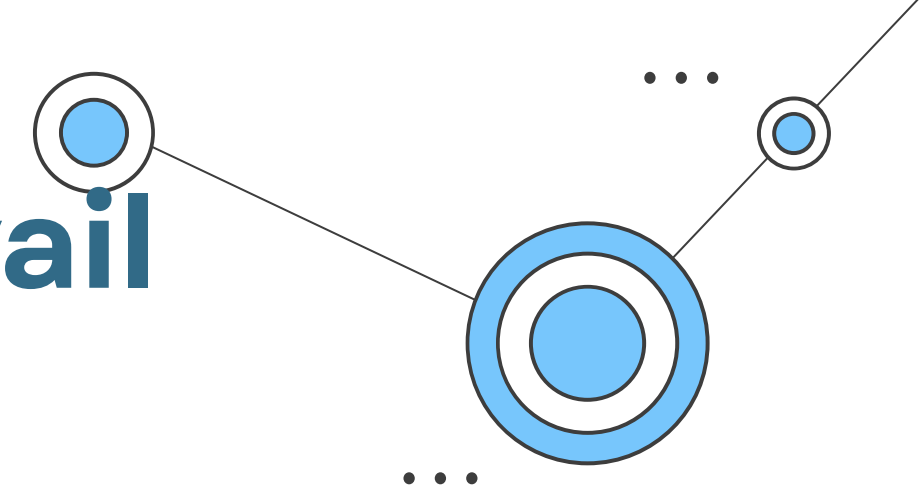
1. Analyse du Besoin et Conception



2. Mise en place de l'Environnement de Travail

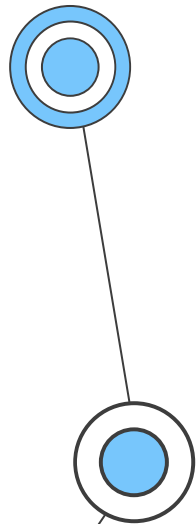
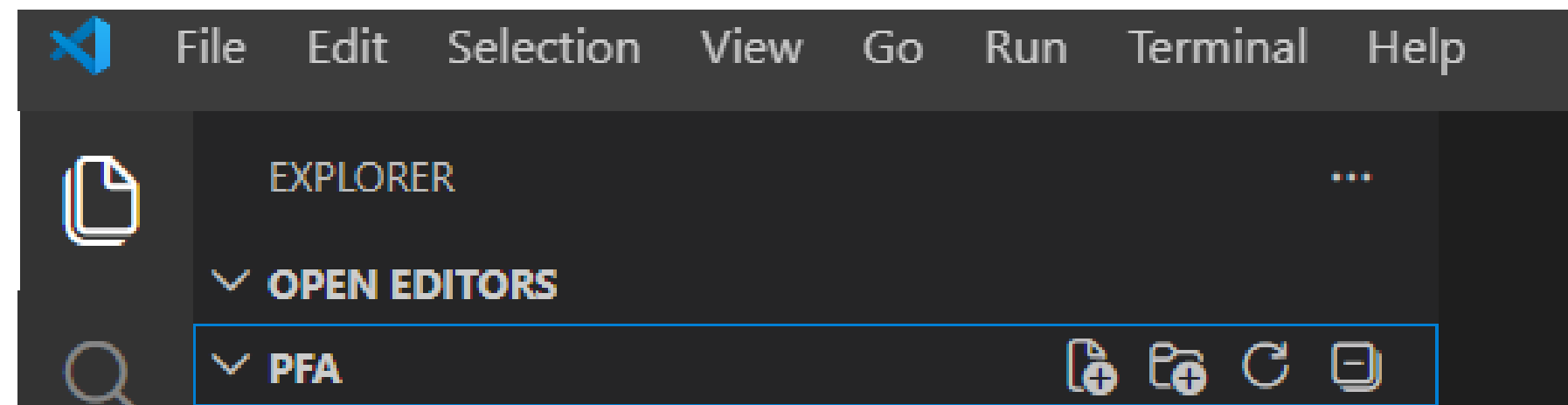
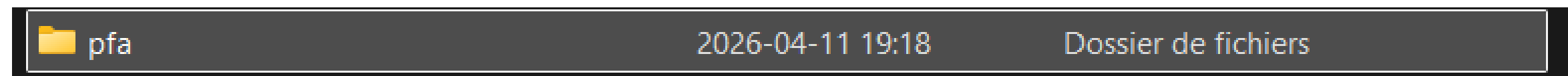


Mise en place de l'Environnement de Travail

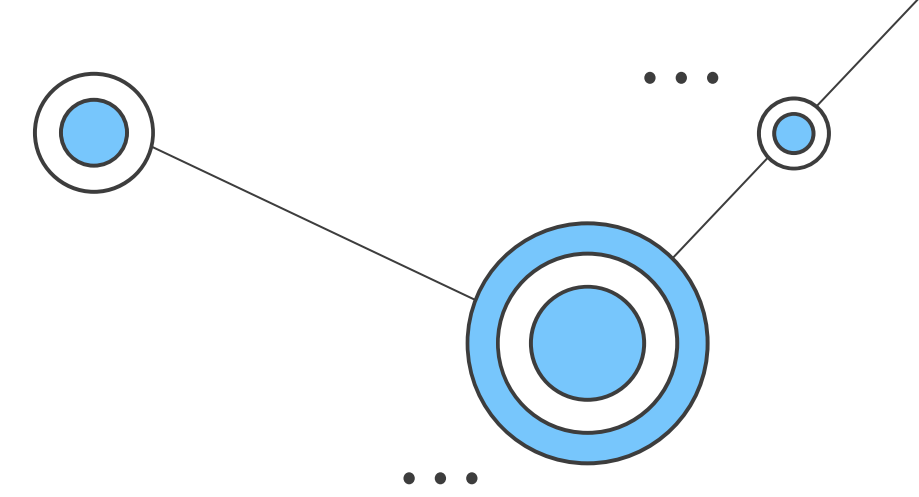


1. Initialisation et Gestion de Version

Le projet est organisé dans un **répertoire local** (un dossier nommé "**PFA**" sur mon ordinateur), que j'ai ouvert et configuré dans l'éditeur **VS Code**. Pour assurer la sauvegarde du code et suivre l'évolution de mon travail



ce dossier est relié à un dépôt distant sur **GitHub**.
L'utilisation de Git permet de conserver un historique de chaque étape du développement, ce qui sécurise le projet contre toute perte de données.



Create a new repository
Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * **takwalabidi** / Repository name *

Great repository names are short and memorable. How about [crispy-carnival](#)?

Description

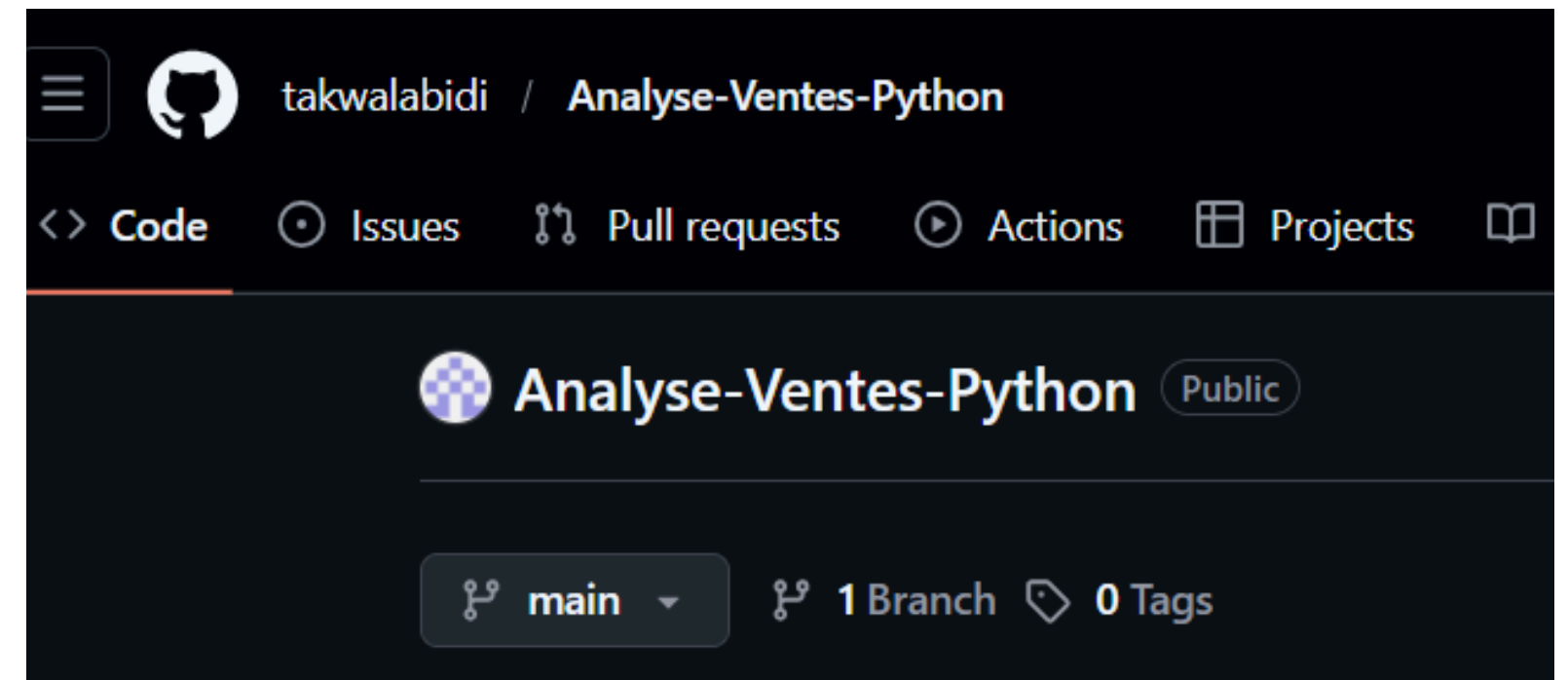
0 / 350 characters

2 Configuration

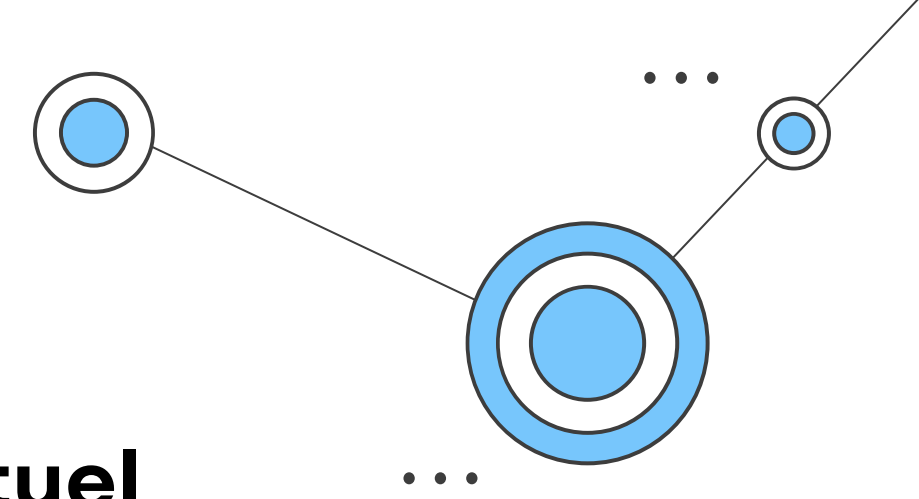
Choose visibility *
Choose who can see and commit to this repository **Public**

Add README
READMEs can be used as longer descriptions. [About READMEs](#) **Off**

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#) **No .gitignore**

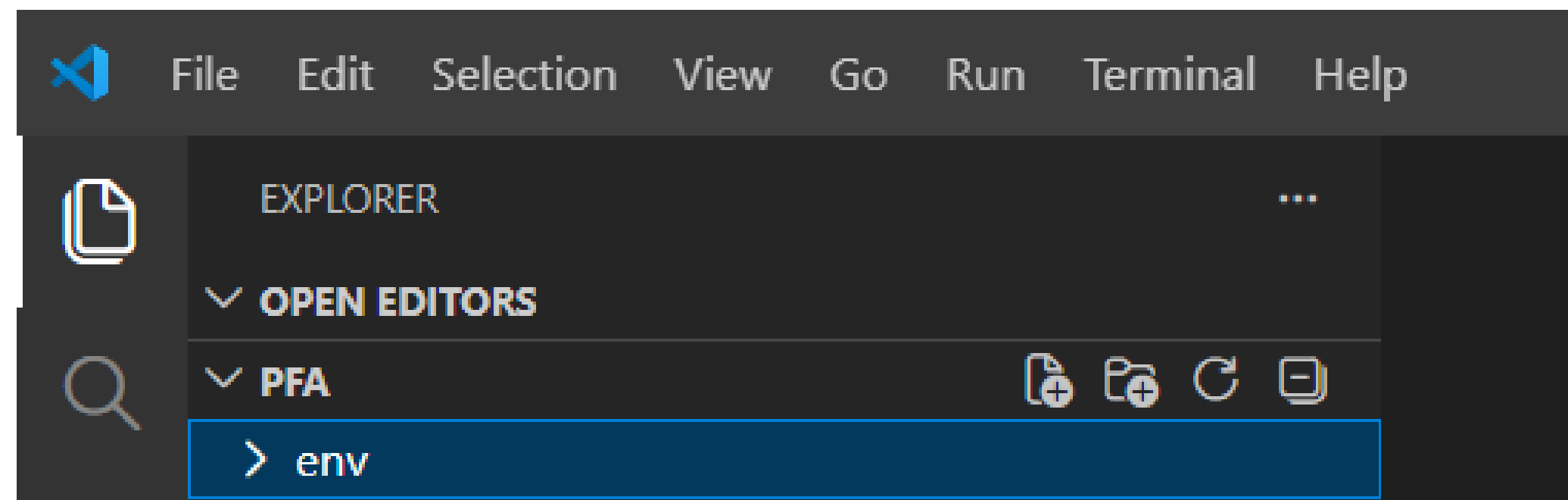
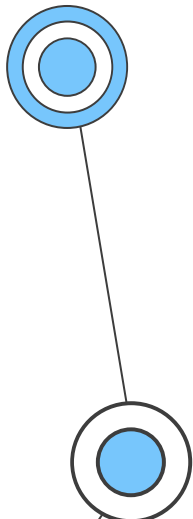


2 .Isolation de l'environnement



Pour travailler de manière isolée, j'ai créé **un environnement virtuel (env)** à l'intérieur de mon dossier de projet. Cela permet de séparer les outils installés pour ce script du reste du système.

```
• PS C:\pfa> python -m venv env  
• PS C:\pfa> .\env\Scripts\activate
```



Ensuite, j'ai utilisé les commandes **pip install pandas** et **pip install matplotlib** pour installer les bibliothèques nécessaires.

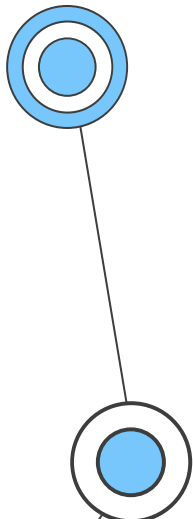
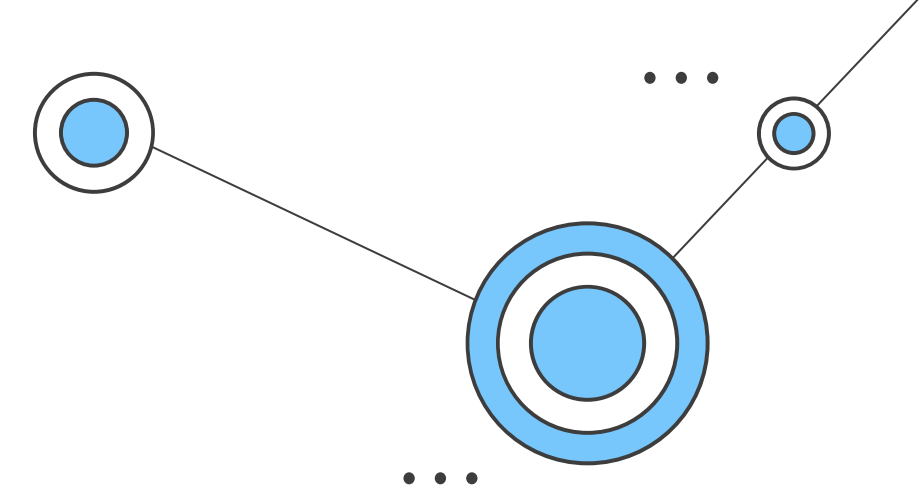
- **Pandas** est utilisé pour la manipulation et l'analyse des données structurées.
- **Matplotlib** permet de générer des graphiques pour visualiser les résultats (comme le CA par produit).

```
• (env) PS C:\pfa> pip install pandas
Collecting pandas
  Using cached pandas-3.0.2-cp313-cp313-win_amd64.whl.metadata
Collecting numpy>=1.26.0 (from pandas)
  Using cached numpy-2.4.4-cp313-cp313-win_amd64.whl.metadata
Collecting python-dateutil>=2.8.2 (from pandas)
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl
Collecting tzdata (from pandas)
  Downloading tzdata-2026.1-py2.py3-none-any.whl.metadata (1.4
```

```
• (env) PS C:\pfa> pip install matplotlib
Collecting matplotlib
  Using cached matplotlib-3.10.8-cp313-cp313-win_amd64.whl.metadata (52 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Using cached contourpy-1.3.3-cp313-cp313-win_amd64.whl.metadata (5.5 kB)
Collecting cyclor>=0.10 (from matplotlib)
  Using cached cyclor-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
  Using cached fonttools-4.62.1-cp313-cp313-win_amd64.whl.metadata (119 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Using cached kiwisolver-1.5.0-cp313-cp313-win_amd64.whl.metadata (5.2 kB)
Requirement already satisfied: numpy>=1.23 in c:\pfa\env\lib\site-packages (from matplotlib)
Collecting packaging>=20.0 (from matplotlib)
  Using cached packaging-26.0-py3-none-any.whl.metadata (3.3 kB)
```

Afin de valider la mise en place de l'environnement, j'ai effectué un contrôle rigoureux des paquets installés.

- **Avant l'installation** : La commande **pip list** montre un environnement vierge
- **Après l'installation** : Une nouvelle vérification confirme la présence de Pandas et Matplotlib, ainsi que leurs dépendances.

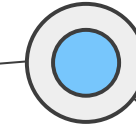
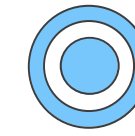


```
(env) PS C:\pfa> pip list
Package Version
-----
pip      25.3
```

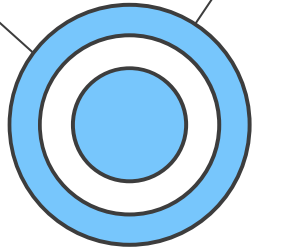
```
(env) PS C:\pfa> pip list
Package Version
-----
contourpy 1.3.3
cyclor 0.12.1
fonttools 4.62.1
kiwisolver 1.5.0
matplotlib 3.10.8
numpy 2.4.4
packaging 26.0
pandas 3.0.2
pillow 12.2.0
pip 25.3
pyparsing 3.3.2
python-dateutil 2.9.0.post0
six 1.17.0
tzdata 2026.1
```

Plan du projet

...



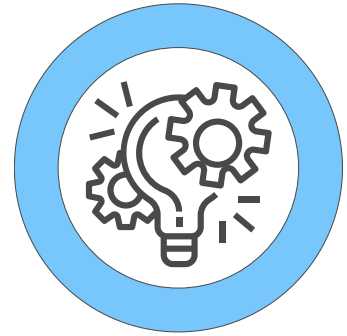
...



...



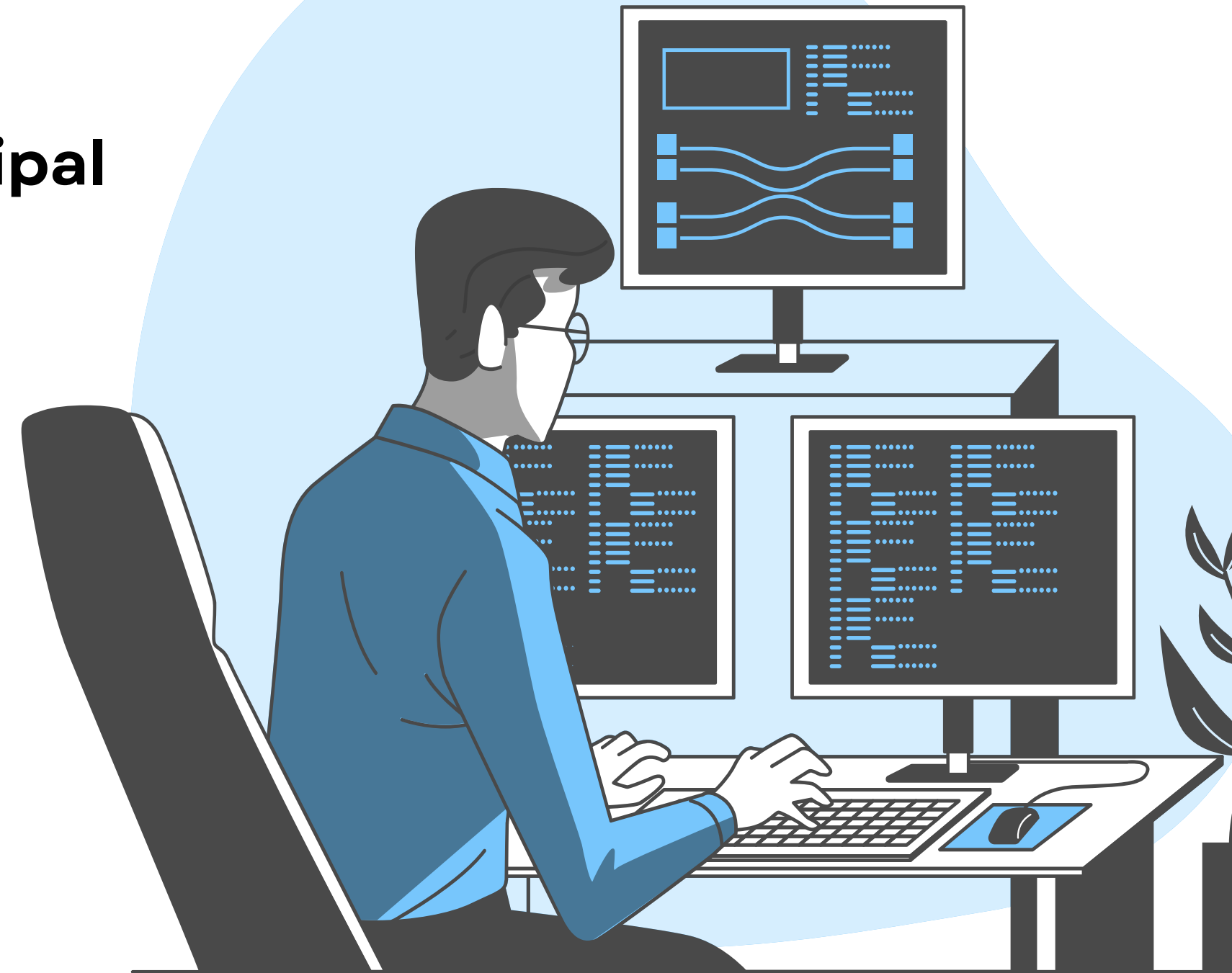
1. Analyse du Besoin et Conception



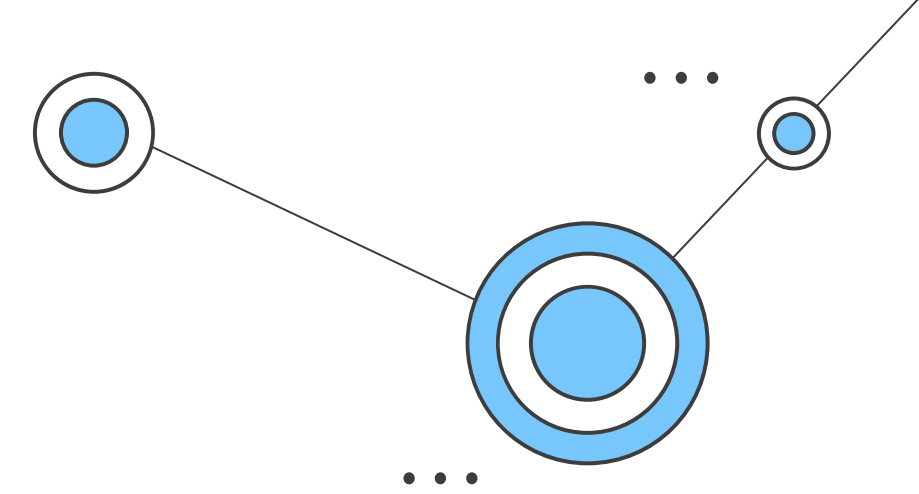
2. Mise en place de l'Environnement de Travail



**3. Développement du Script Principal
(main.py)**



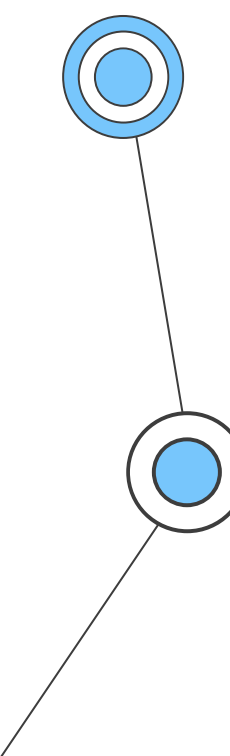
Développement du Script Principal (main.py)



1. Configuration et Initialisation des Données

Pour débiter le script : **importation des bibliothèques**

- **Pandas (import pandas as pd)** : Sélectionnée pour sa puissance dans la lecture du fichier CSV et le calcul automatisé des données financières.
- **Matplotlib (import matplotlib.pyplot as plt)** : Utilisée spécifiquement pour la partie visuelle du projet, permettant de traduire les résultats chiffrés en graphiques clairs et explicites.



```
main.py > ...  
1  import pandas as pd  
2  import matplotlib.pyplot as plt
```


Pour simuler un flux de données réel, le script définit d'abord **un dictionnaire** contenant les colonnes **ID, Prix, Quantité et Remise**.

Ces données sont ensuite converties en un **DataFrame** Pandas pour être structurées sous forme de tableau.

Enfin, la méthode **to_csv()** est utilisée pour exporter automatiquement ces informations dans le fichier **ventes.csv**

```
# 1. GÉNÉRATION DES DONNÉES
# On crée directement les données en mémoire

data = {
    "ID": [101, 102, 103],
    "Prix": [15.0, 25.0, 10.0],
    "Quantite": [3, 2, 5],
    "Remise": [10, 5, 0]
}

df = pd.DataFrame(data)

# Sauvegarde automatique du fichier CSV
df.to_csv("ventes.csv", index=False)
print(" Fichier ventes.csv généré automatiquement !")
```


2. Logique de Calcul et Transformation

Une fois les données chargées, le script exécute une série de transformations mathématiques pour enrichir le tableau :

- **Calcul du CA Brut** : Création d'une colonne calculée en multipliant le prix par la quantité.
-
- **Traitement des Remises** : Application des pourcentages de remise pour obtenir le CA Net.
-
- Calcul de la TVA : Génération d'une colonne dédiée appliquant le taux de 20 % sur le montant net, conformément aux exigences

```

# 2. Calcul du Chiffre d'Affaires Brut (Prix x Quantité)
df["CA_Brut"] = df["Prix"] * df["Quantite"]
print(df["CA_Brut"])

# 3. Application des remises (%) → CA Net
df["CA_Net"] = df["CA_Brut"] * (1 - df["Remise"] / 100)
print(df["CA_Net"])

# 4. Calcul de la TVA (20%) sur le CA Net
df["TVA"] = df["CA_Net"] * 0.20
print(df["TVA"])

# 5. Calcul du CA Total de l'entreprise
ca_total = df["CA_Net"].sum()
print("CA Total =", ca_total)

#6. Identifier l'ID du produit avec le plus grand bénéfice (CA Net max)
id_best_product = df.loc[df["CA_Net"].idxmax(), "ID"]
print("Produit avec le plus grand bénéfice :", id_best_product)

# 7. Export du fichier final
df.to_csv("resultats_final.csv", index=False)
print("Fichier exporté avec succès !")

```

Calcul du CA Brut : Le script crée une nouvelle colonne **CA_Brut** en effectuant une opération vectorisée (Prix * Quantité). Cette méthode traite toutes les lignes du fichier instantanément

Application de la remise : Pour obtenir le **CA Net**, nous appliquons la formule mathématique. Cela permet de convertir le chiffre entier de la colonne "Remise" en un coefficient réducteur.

Détermination de la TVA : Conformément à l'énoncé, la TVA est calculée en appliquant un taux de **20 %** (soit un multiplicateur de 0.20) sur le montant net précédemment obtenu.

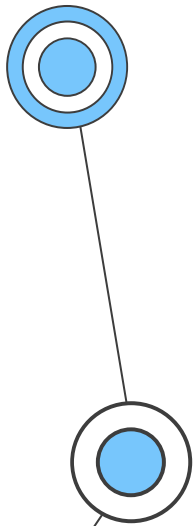
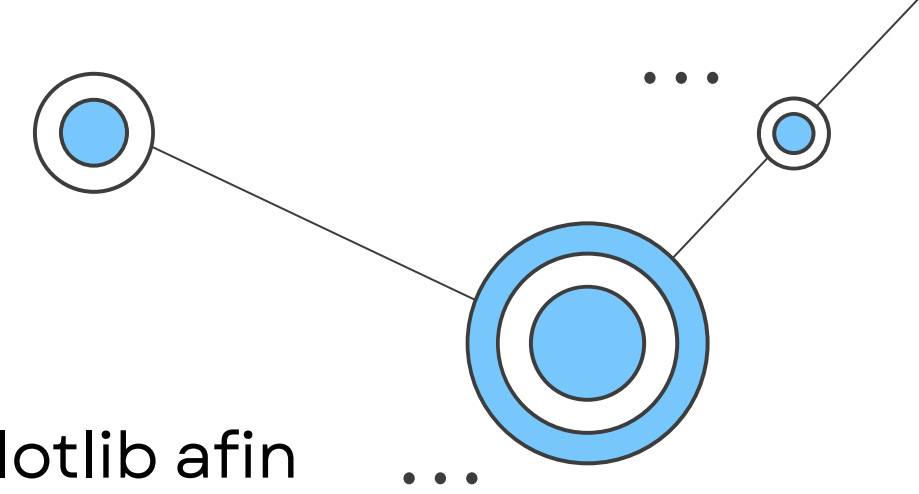
- La fonction **.sum()** est utilisée pour additionner toutes les valeurs de la colonne **CA_Net** et afficher le revenu global de l'entreprise.
- La méthode **.idxmax()** identifie l'indice de la ligne ayant le montant le plus élevé, nous permettant de récupérer l'ID du produit le plus rentable

Exportation des résultats : Sauvegarde automatique de l'analyse dans un fichier final nommé **resultats_final.csv** pour garantir la persistance des données calculées

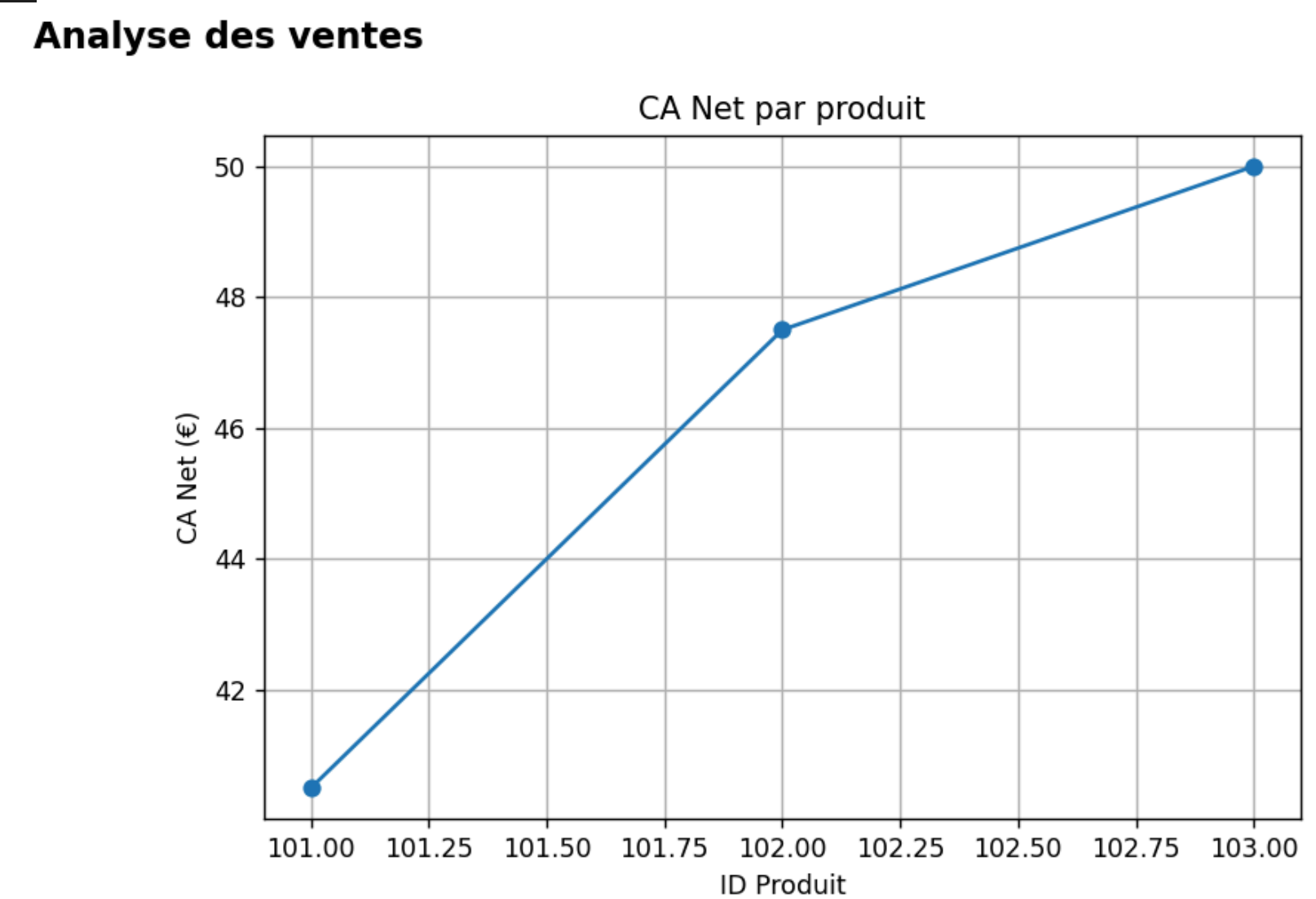
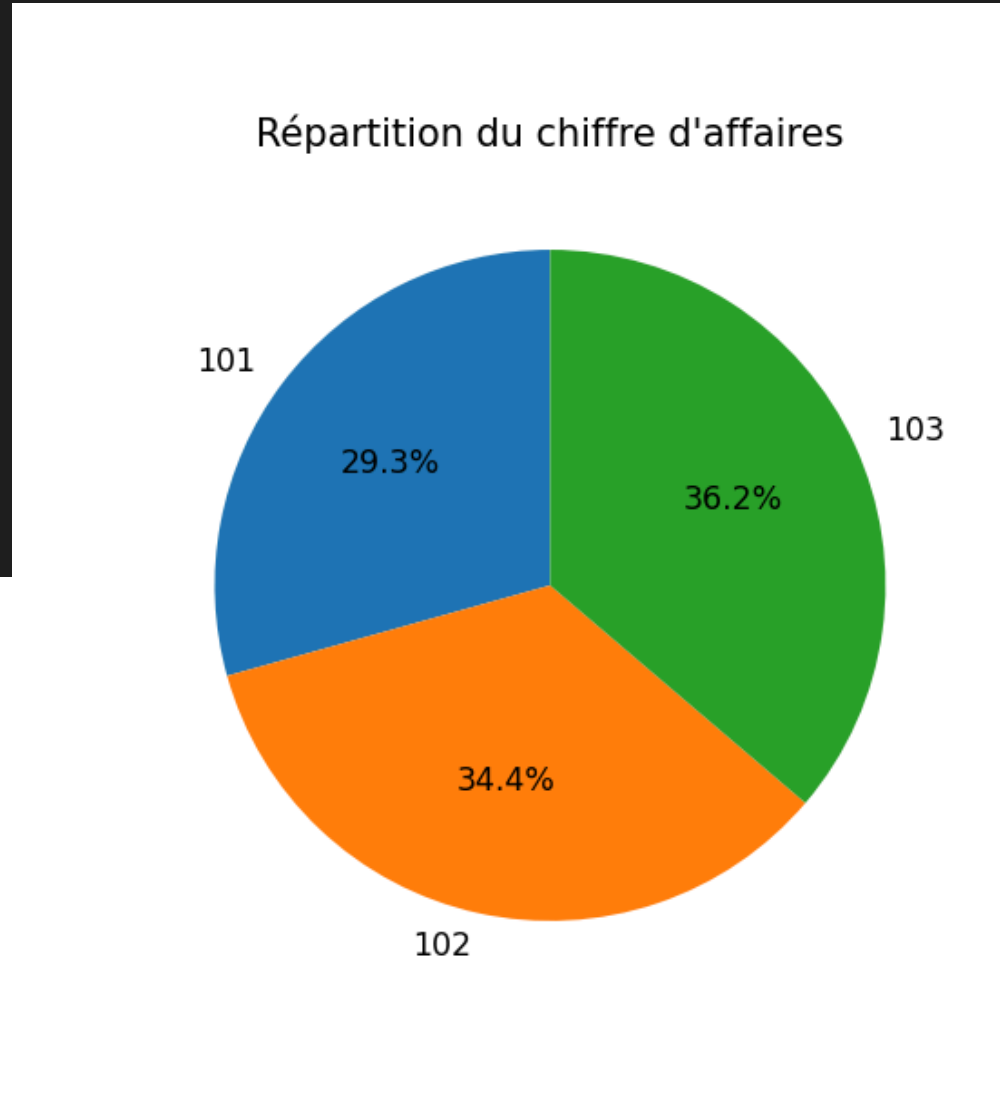
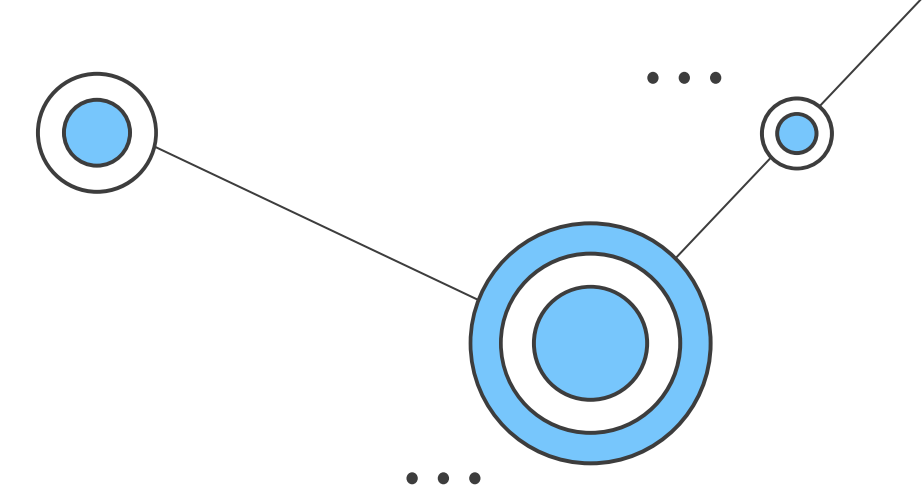
3. Visualisation Graphique des Résultats

Pour rendre l'analyse plus parlante, nous avons utilisé la bibliothèque Matplotlib afin de générer deux graphiques complémentaires :

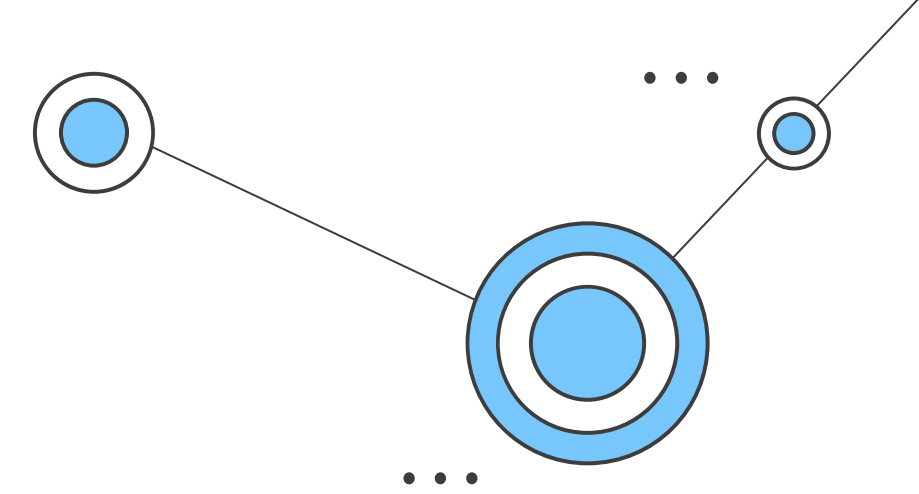
- **Configuration de la zone de dessin : `plt.subplots(1, 2)`** permet de créer une figure contenant deux graphiques côte à côte pour une comparaison directe.
- **Graphique 1 (Circulaire)** : Utilisation de `axes[0].pie()` pour visualiser la répartition du chiffre d'affaires net par produit (ID), avec un affichage des pourcentages automatisé.
- **Graphique 2 (Linéaire)** : Utilisation de `axes[1].plot()` pour tracer l'évolution du chiffre d'affaires net, incluant des marqueurs pour identifier chaque point de données précisément.
- **Finalisation et Export :**
 - **`plt.tight_layout()`** ajuste automatiquement l'espacement pour éviter les chevauchements.
 - **`plt.savefig("graphiques_ventes.png")`** sauvegarde le visuel final sous format image pour une utilisation externe.
 - **`plt.show()`** affiche les graphiques à l'écran lors de l'exécution



```
44 # 8. VISUALISATION
45
46 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
47
48 # Titre général
49 fig.suptitle("Analyse des ventes", fontsize=14, fontweight="bold")
50
51 # --- Graphique 1 : Cercle (Camembert)
52 axes[0].pie(
53     df["CA_Net"],
54     labels=df["ID"],
55     autopct="%1.1f%%",
56     startangle=90
57 )
58 axes[0].set_title("Répartition du chiffre d'affaires")
59
60 # --- Graphique 2 : Graphique avec axes (courbe simple)
61 axes[1].plot(df["ID"], df["CA_Net"], marker='o')
62 axes[1].set_title("CA Net par produit")
63 axes[1].set_xlabel("ID Produit")
64 axes[1].set_ylabel("CA Net (€)")
65 axes[1].grid()
66
67 plt.tight_layout()
68 plt.savefig("graphiques_ventes.png")
69 plt.show()
70
71 print(" Graphiques générés !")
```



4. Finalisation et Exportation



La dernière phase du script assure la persistance des données.

Après avoir identifié le produit ayant généré le plus gros bénéfice, l'ensemble du tableau enrichi est exporté vers un nouveau fichier nommé **resultats_final.csv**.

resultats_final.csv X

resultats_final.csv > data

1	ID,Prix,Quantite,Remise,CA_Brut,CA_Net,TVA
2	101,15,0.3,10,4.5,4.05,0.81
3	102,25,0.2,5,5.0,4.75,0.9500000000000001
4	103,10,0.5,0,5.0,5.0,1.0
5	

```
(env) PS C:\pfa> & c:\pfa\env\Scripts\python.exe c:/pfa/main2.py
Fichier ventes.csv généré automatiquement !
0    45.0
1    50.0
2    50.0
Name: CA_Brut, dtype: float64
0    40.5
1    47.5
2    50.0
Name: CA_Net, dtype: float64
0     8.1
1     9.5
2    10.0
Name: TVA, dtype: float64
CA Total = 138.0
Produit avec le plus grand bénéfice : 103
Fichier exporté avec succès !
Graphiques générés !
```


Pour finaliser la portabilité du projet, j'ai généré un fichier **requirements.txt** à l'aide de la commande : **pip freeze > requirements.txt**.

Ce fichier liste précisément toutes les bibliothèques utilisées (**Pandas, Matplotlib, etc.**) ... ainsi que leurs versions exactes.

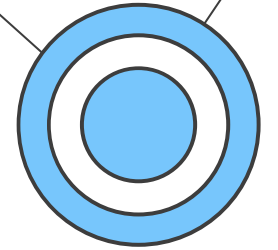
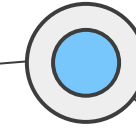
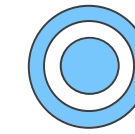
Cela permet à **un autre utilisateur** d'installer tout **l'environnement** de travail d'un seul coup avec la commande **pip install -r requirements.txt**, garantissant que le script fonctionnera exactement de la même manière sur n'importe quelle machine.

```
(env) PS C:\pfa> pip freeze > requirements.txt
```

```
requirements.txt
1  contourpy==1.3.3
2  cycler==0.12.1
3  fonttools==4.62.1
4  kiwisolver==1.5.0
5  matplotlib==3.10.8
6  numpy==2.4.4
7  packaging==26.0
8  pandas==3.0.2
9  pillow==12.2.0
10 pyparsing==3.3.2
11 python-dateutil==2.9.0.post0
12 six==1.17.0
13 tzdata==2026.1
14
```


Plan du projet

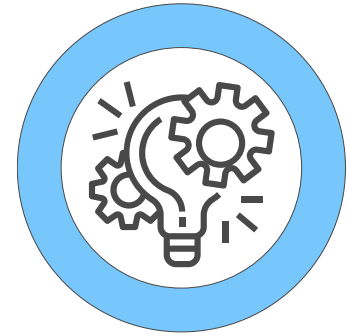
...



...



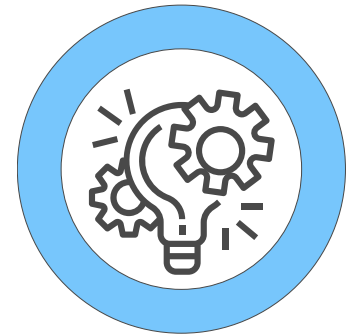
1. Analyse du Besoin et Conception



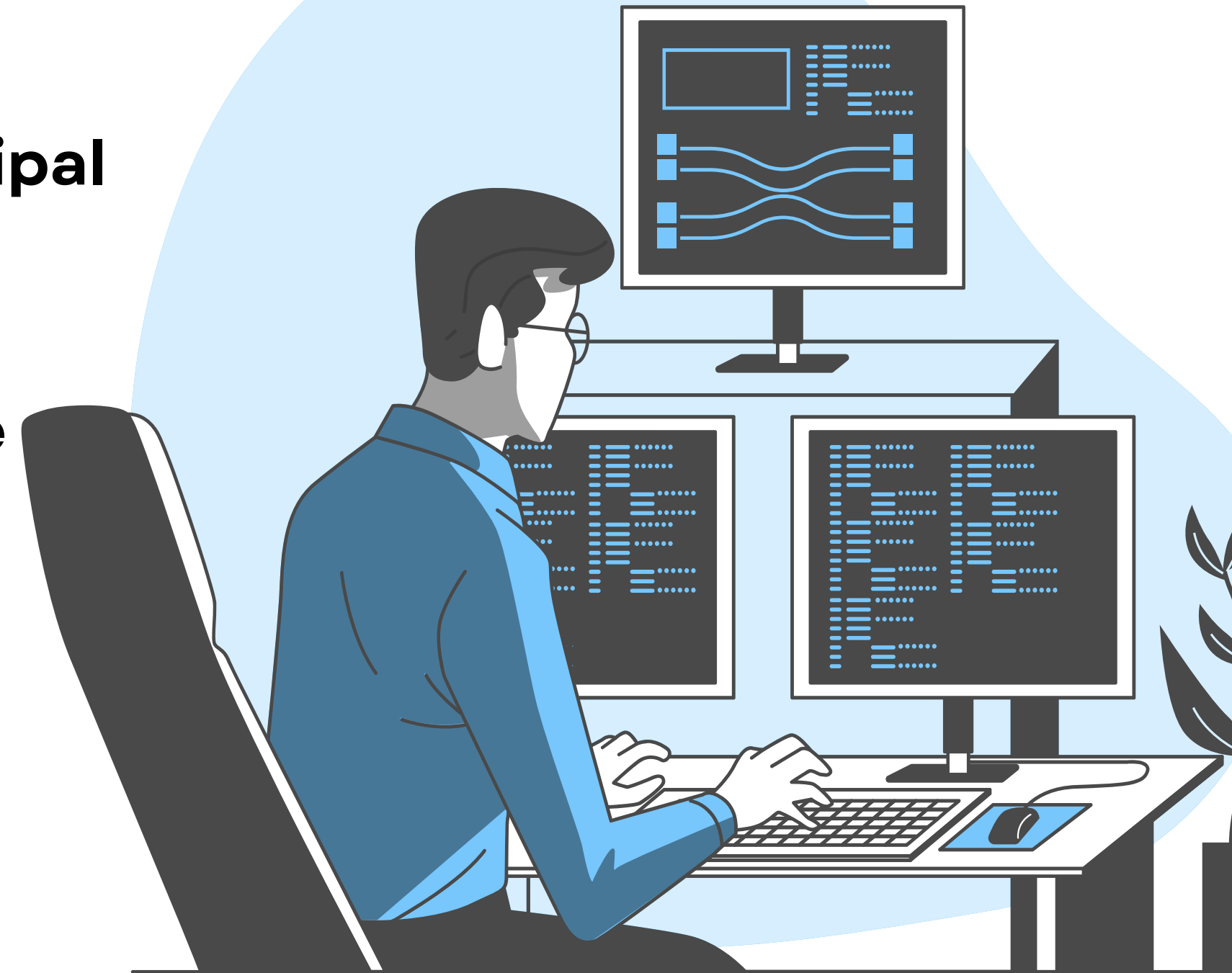
2. Mise en place de l'Environnement de Travail



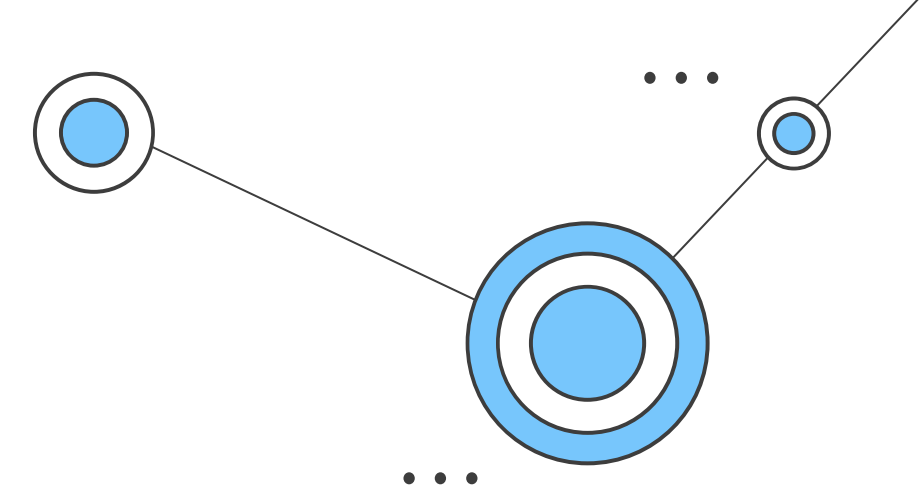
**3. Développement du Script Principal
(main.py)**



**4. Tests de Robustesse et "Lecture
Dynamique"**



Tests de Robustesse et "Lecture Dynamique"



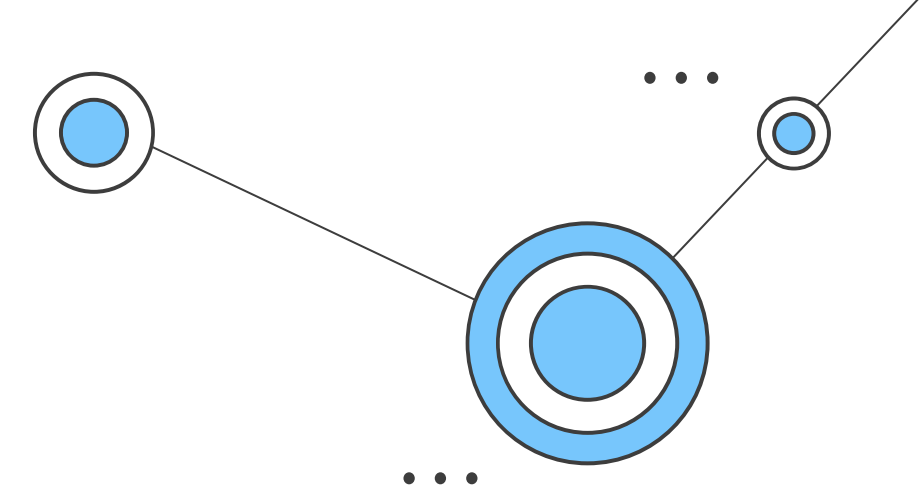
1. Génération de données massives

Pour tester la robustesse et la performance de l'application, nous avons automatisé la création **d'un jeu de données** plus conséquent :

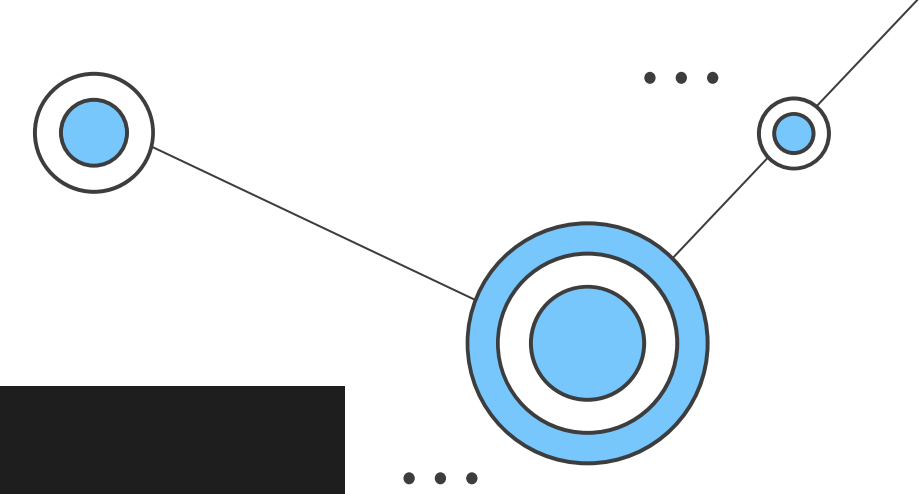
- **Montée en charge** : Le script génère désormais **1 000 lignes** de données uniques (produits) au lieu de quelques entrées manuelles.
- **Utilisation de la bibliothèque random** : Cette bibliothèque est sollicitée pour produire des valeurs variées et réalistes :
 - Prix : Générés aléatoirement entre 5 et 100 avec deux décimales.
 - Quantité : Valeurs entières aléatoires comprises entre 1 et 10.
 - Remise : Attribution aléatoire d'un taux parmi une liste prédéfinie (0, 5, 10, 15, 20).
- **Automatisation** : Le processus crée instantanément un fichier ventes.csv structuré, permettant de simuler une base de données de vente réelle pour l'analyse financière.

Tests de Robustesse et "Lecture Dynamique"

1. Génération de données massives



```
generer_data2.py > ...
1  import pandas as pd
2  import matplotlib.pyplot as plt
3  import random
4
5  # 1. GÉNÉRATION DE 1000 PRODUITS UNIQUES
6
7  nb_lignes = 1000
8
9  data = {
10     "ID": list(range(1, nb_lignes + 1)), # 1000 ID uniques
11     "Prix": [round(random.uniform(5, 100), 2) for _ in range(nb_lignes)],
12     "Quantite": [random.randint(1, 10) for _ in range(nb_lignes)],
13     "Remise": [random.choice([0, 5, 10, 15, 20]) for _ in range(nb_lignes)]
14 }
15
16 df = pd.DataFrame(data)
17
18 # Sauvegarde CSV
19 df.to_csv("ventes.csv", index=False)
20 print("Fichier ventes.csv (1000 produits uniques) généré !")
21
```



```
# 2. CALCULS
```

```
df["CA_Brut"] = df["Prix"] * df["Quantite"]  
df["CA_Net"] = df["CA_Brut"] * (1 - df["Remise"] / 100)  
df["TVA"] = df["CA_Net"] * 0.20
```

```
# 3. ANALYSE
```

```
ca_total = df["CA_Net"].sum()
```

```
# Ici chaque produit est unique → pas besoin de groupby
```

```
top_produit = df.loc[df["CA_Net"].idxmax()]
```

```
print(f"\n CA Total = {ca_total:.2f} €")
```

```
print(f" Meilleur produit : ID {top_produit['ID']} → {top_produit['CA_Net']:.2f} €")
```

```
# 4. EXPORT FINAL
```

```
df.to_csv("resultats_final.csv", index=False)
```

```
print(" resultats_final.csv exporté !")
```

```
# 5. VISUALISATION
```

```

# 5. VISUALISATION
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Titre général
fig.suptitle("Analyse des ventes", fontsize=14, fontweight="bold")

# --- Graphique 1 : Camembert ---
# On ne peut pas afficher 1000 parts → on prend TOP 10
top10 = df.sort_values(by="CA_Net", ascending=False).head(10)

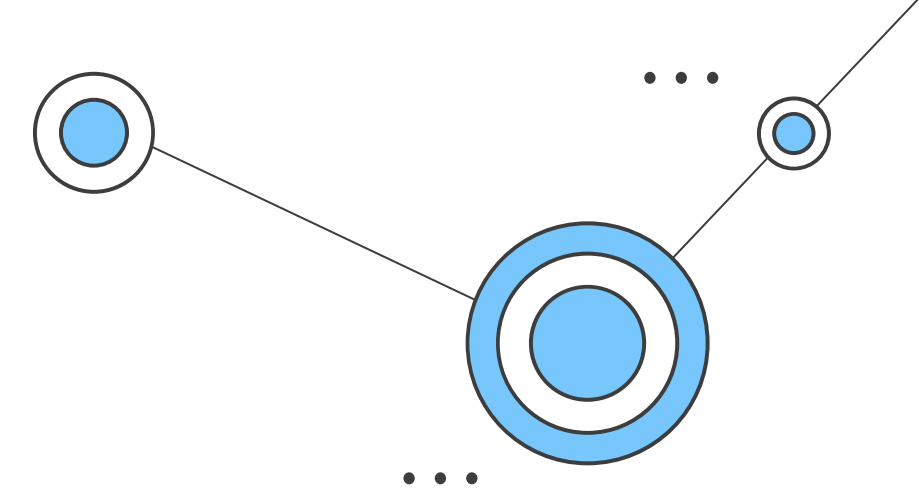
axes[0].pie(
    top10["CA_Net"],
    labels=top10["ID"],
    autopct="%1.1f%%",
    startangle=90
)
axes[0].set_title("Top 10 produits (CA Net)")

# --- Graphique 2 : Courbe ---
axes[1].plot(df["ID"], df["CA_Net"])
axes[1].set_title("CA Net des produits")
axes[1].set_xlabel("ID Produit")
axes[1].set_ylabel("CA Net (€)")
axes[1].grid()

plt.tight_layout()
plt.savefig("graphiques_ventes.png")
plt.show()

print(" Graphiques générés !")

```



2.Validation du dynamisme

Le test **a confirmé** que le script est entièrement dynamique.

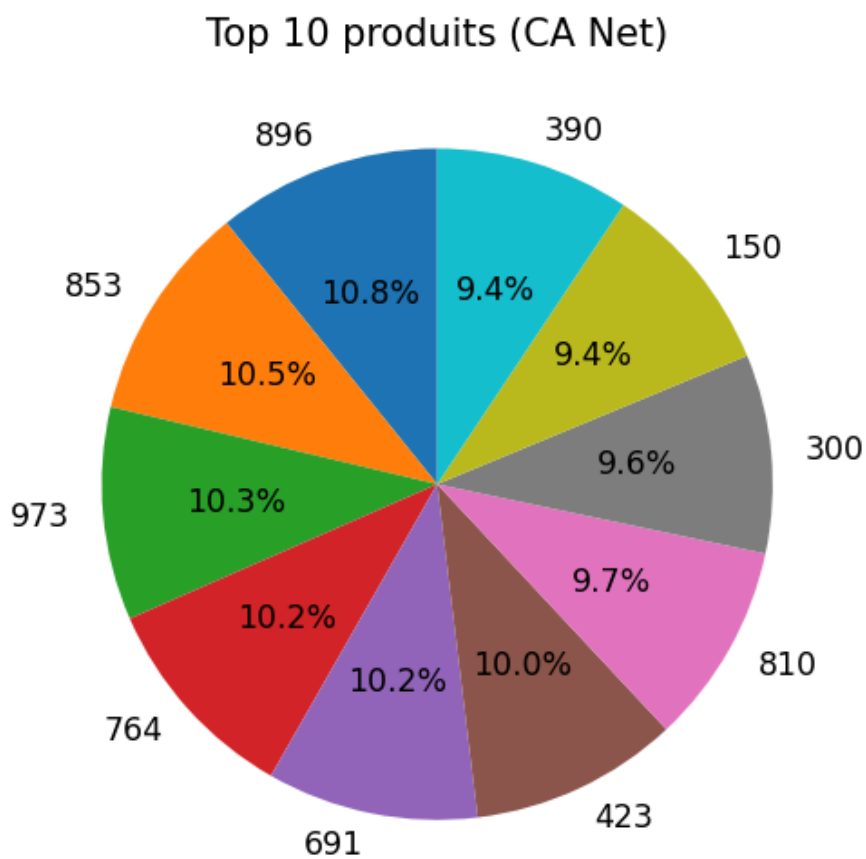
Grâce à l'utilisation **de Pandas**, le code s'adapte automatiquement au nombre de lignes **du fichier CSV** sans aucune modification manuelle.

Le calcul du CA Total et la génération du graphique

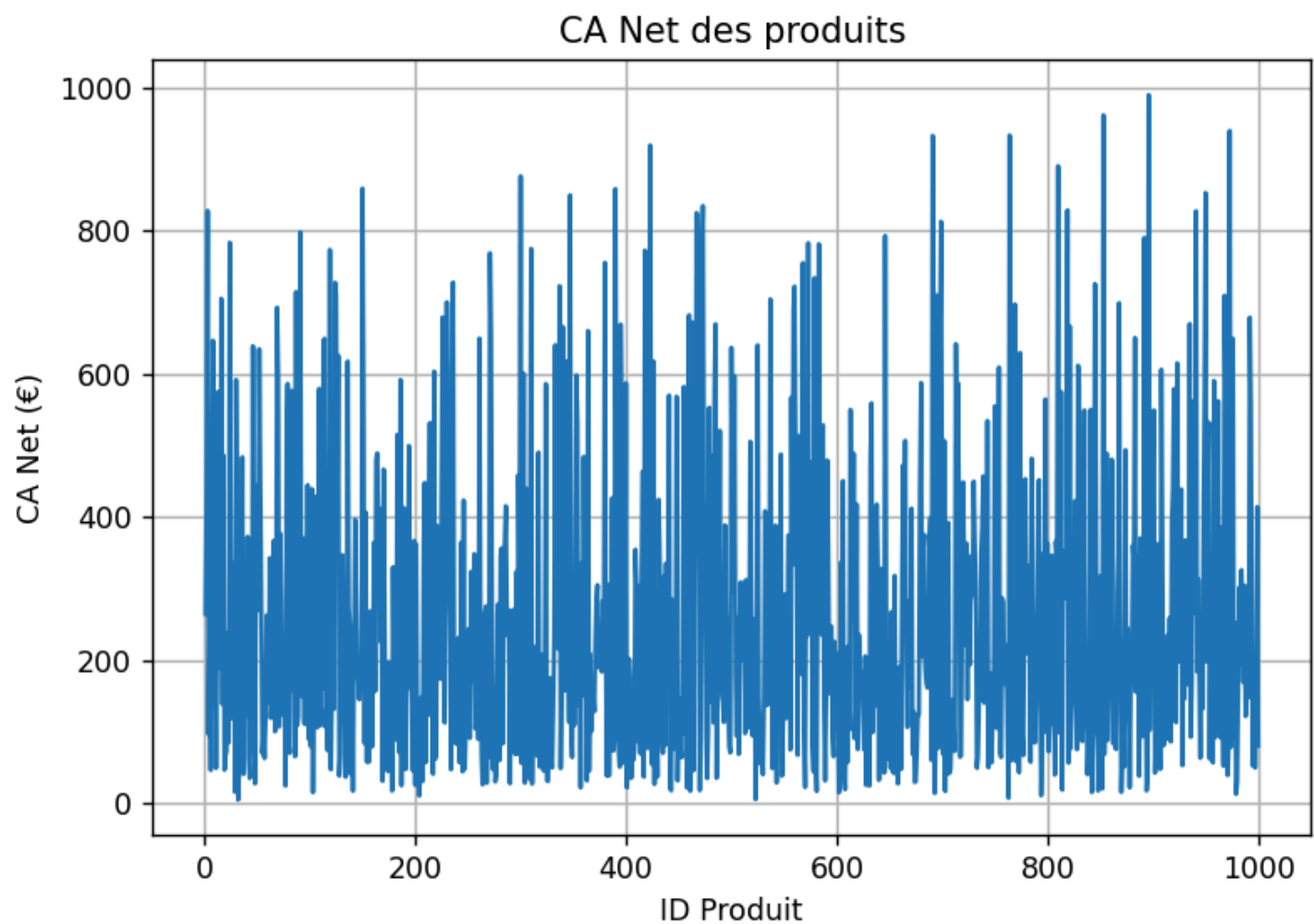
Matplotlib s'exécute instantanément, prouvant ainsi la fiabilité et la scalabilité de la solution développée.

```
PS C:\pfa> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned)
(env) PS C:\pfa> & c:\pfa\env\Scripts\python.exe c:/pfa/generer_data.py
Fichier ventes.csv (1000 produits uniques) généré !

CA Total = 269660.83 €
Meilleur produit : ID 283.0 → 996.50 €
resultats_final.csv exporté !
```

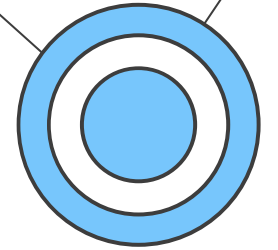
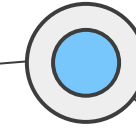
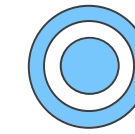


Analyse des ventes



Plan du projet

...

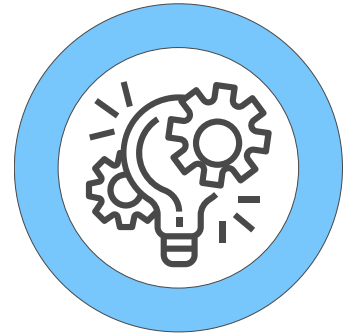


...

...



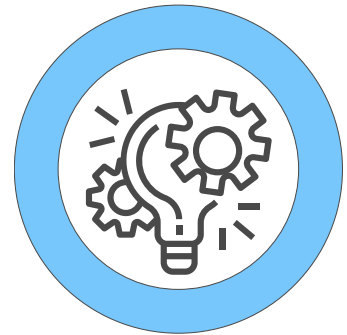
1. Analyse du Besoin et Conception



2. Mise en place de l'Environnement de Travail



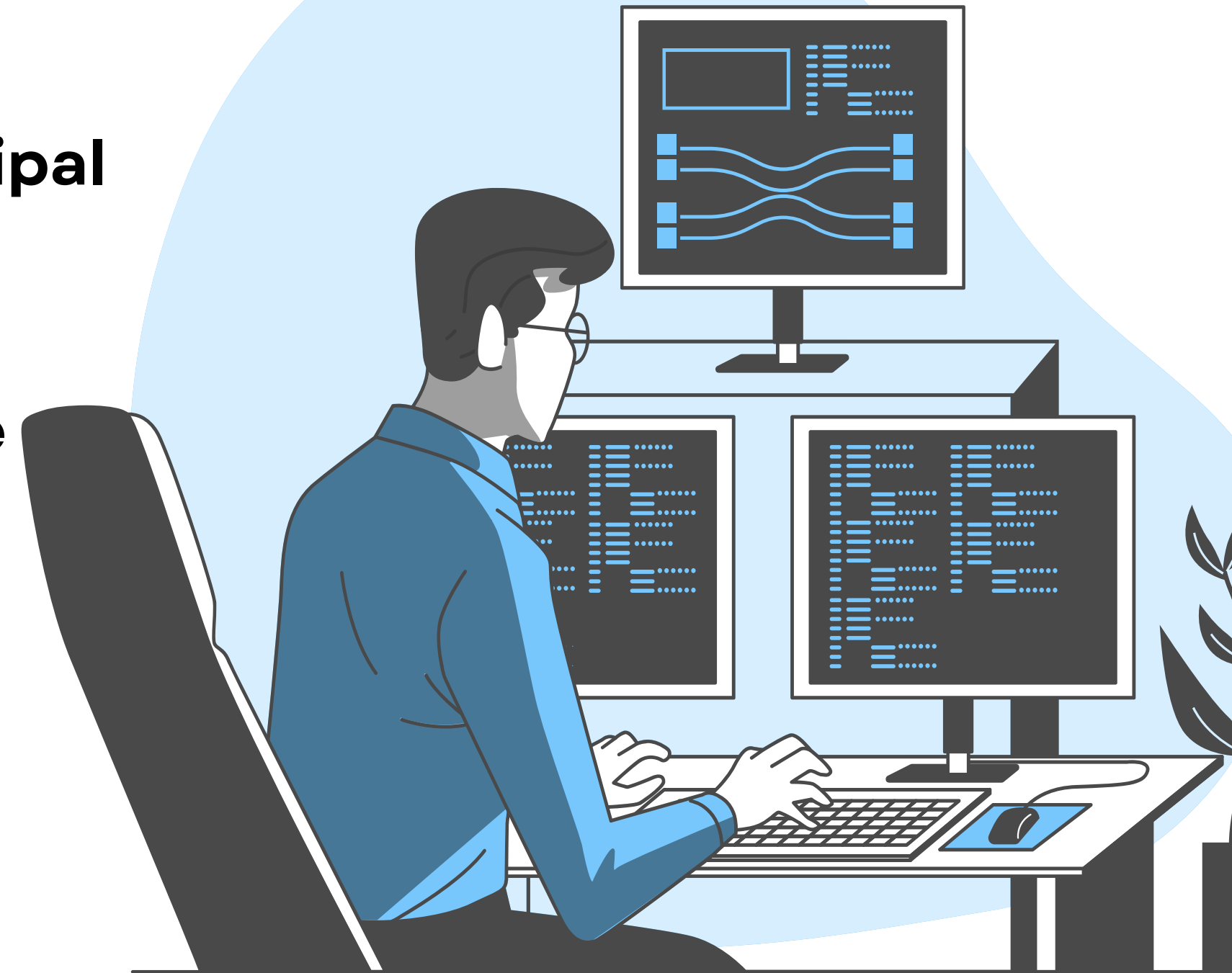
**3. Développement du Script Principal
(main.py)**



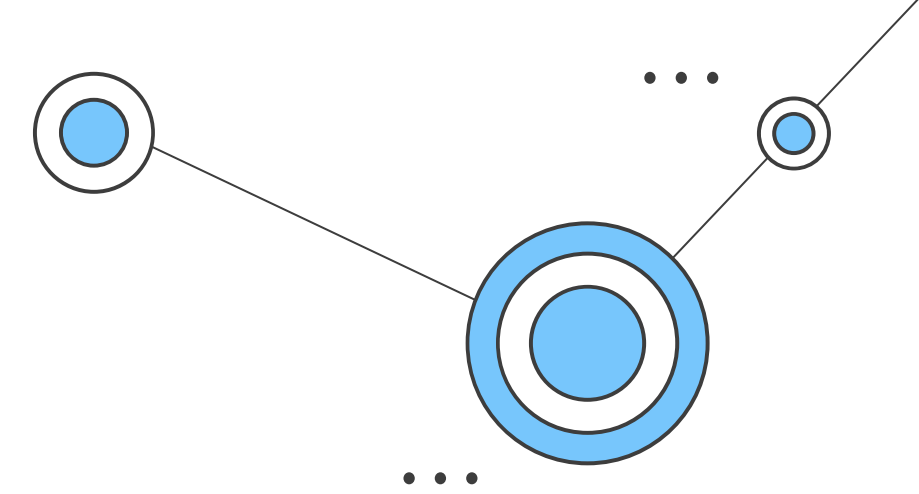
**4. Tests de Robustesse et "Lecture
Dynamique"**



5. Documentation et Versioning



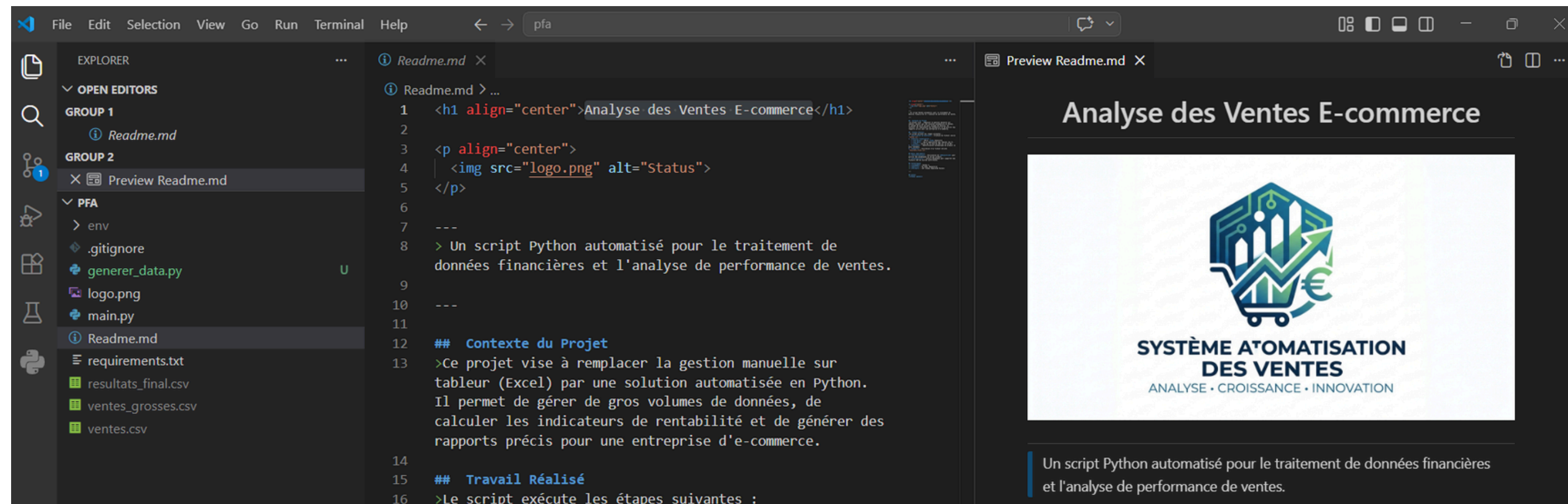
Documentation et Versioning



1. Rédaction du README

Un fichier **README.md** a été rédigé à la racine du projet pour servir de documentation technique.

Ce guide explique clairement l'objectif du script, les prérequis d'installation et la marche à suivre pour exécuter l'analyse. Pour rendre le projet plus professionnel, j'ai également conçu **un logo personnalisé** intégré à la documentation, renforçant l'identité visuelle du logiciel.

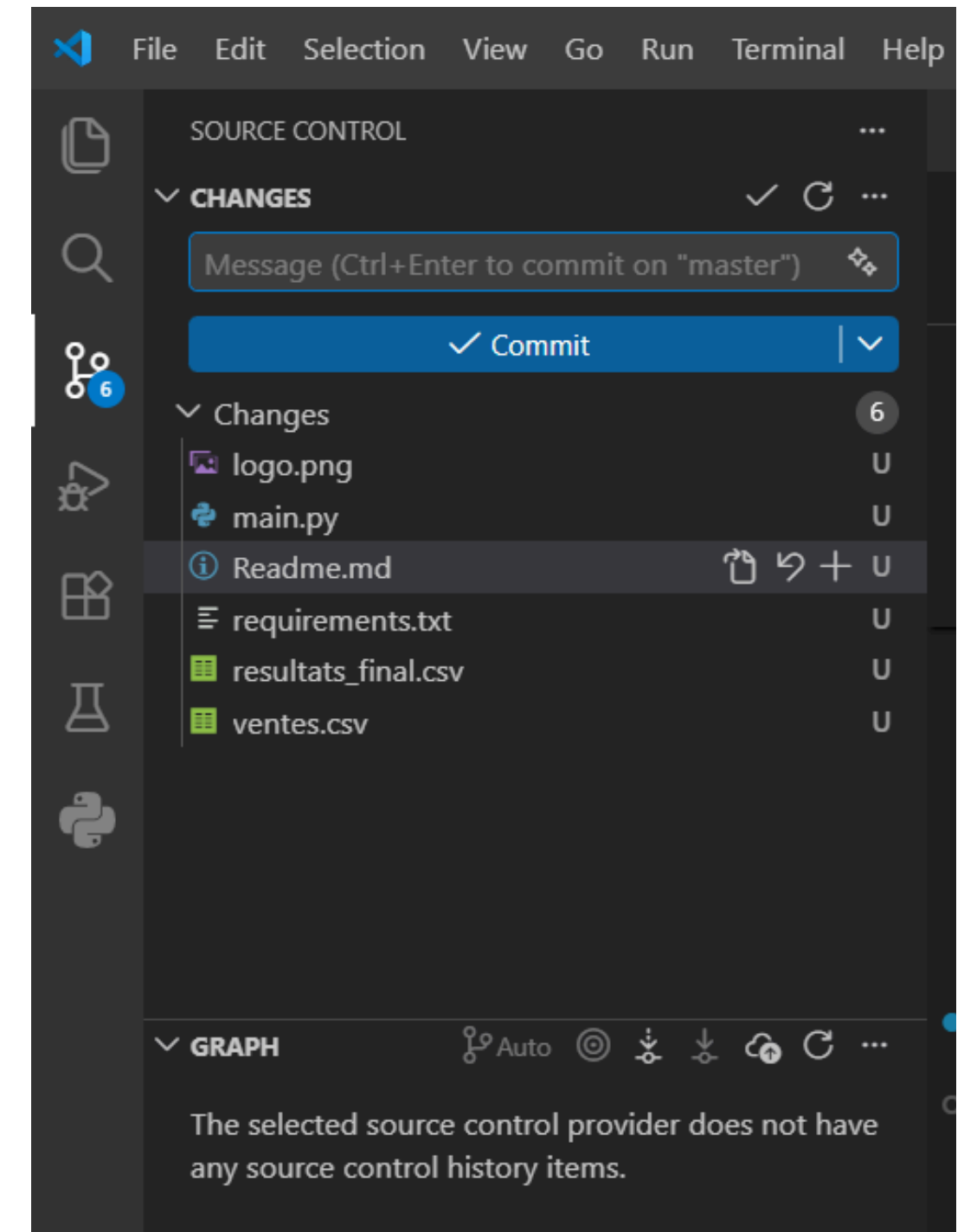


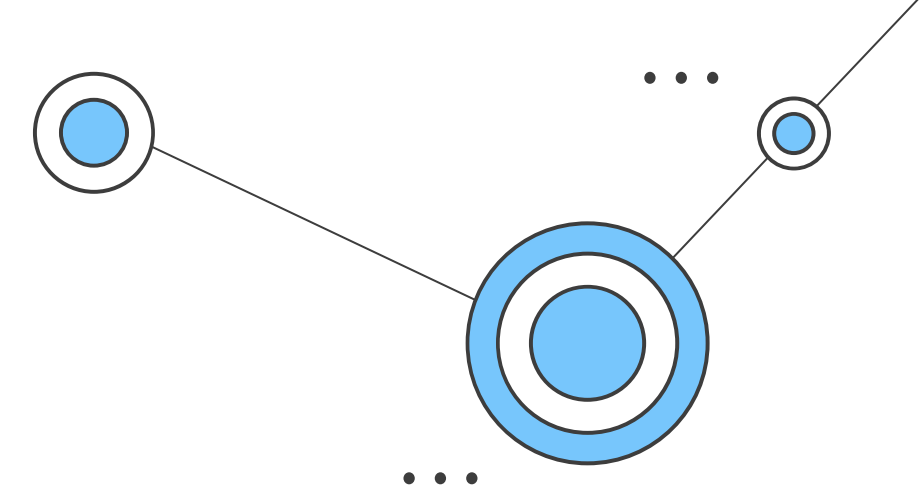
2 .Synchronisation avec GitHub

Pour sauvegarder mon travail, j'ai utilisé le cycle standard de **Git** via trois commandes clés :

- **git add .** : Pour préparer tous mes fichiers (code, CSV, logo) avant la sauvegarde.
- **git commit -m "Message"** : Pour enregistrer une étape précise de mon projet (ex: "Initialisation", "Correction calculs", "Ajout graphique"). Cela crée un historique de mon travail.
- **git push** : Pour envoyer toutes ces modifications sur mon dépôt GitHub en ligne.

Grâce à ce processus, mon code est en sécurité sur le Cloud et je peux retrouver n'importe quelle version précédente de mon projet en cas d'erreur





```

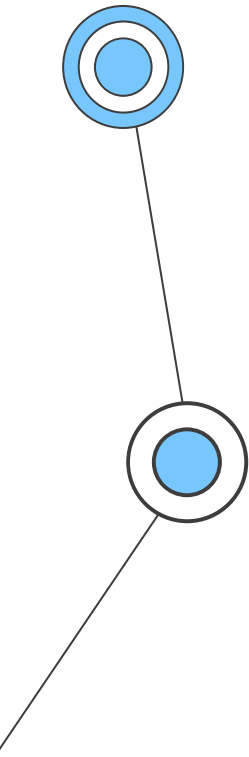
  (env) PS C:\pfa> pip freeze > requirements.txt
  • (env) PS C:\pfa> git init
    Initialized empty Git repository in C:/pfa/.git/
  • (env) PS C:\pfa> git add .
  • (env) PS C:\pfa> git commit -m "Initialisation du projet d'analyse e-commerce"
    [master (root-commit) 340f5d1] Initialisation du projet d'analyse e-commerce
    6 files changed, 81 insertions(+)
    create mode 100644 Readme.md
    create mode 100644 logo.png
    create mode 100644 main.py
    create mode 100644 requirements.txt
    create mode 100644 resultats_final.csv
    create mode 100644 ventes.csv
  ○ (env) PS C:\pfa>

```

```

  (env) PS C:\pfa> git remote add origin https://github.com/takwalabidi/Analyse-V
  (env) PS C:\pfa> git branch -M main
  (env) PS C:\pfa> git push -u origin main
    info: please complete authentication in your browser...
    Enumerating objects: 11, done.
    Counting objects: 100% (11/11), done.
    Delta compression using up to 8 threads
    Compressing objects: 100% (11/11), done.
    Writing objects: 100% (11/11), 766.40 KiB | 17.03 MiB/s, done.
    Total 11 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
    remote: Resolving deltas: 100% (1/1), done.
    To https://github.com/takwalabidi/Analyse-Ventes-Python.git
     * [new branch]      main -> main
    branch 'main' set up to track 'origin/main'.
  ○ (env) PS C:\pfa>

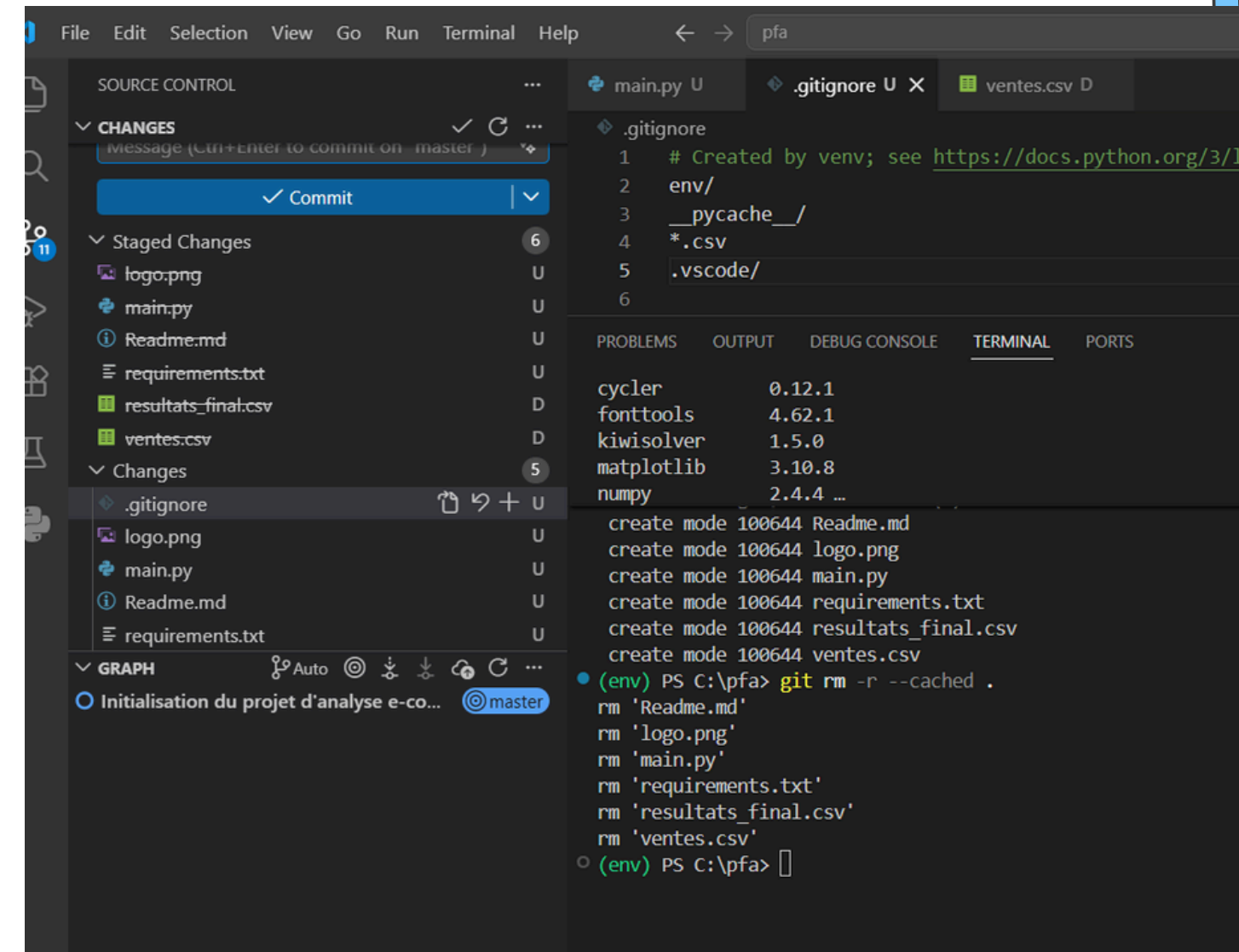
```



3 .Sécurité et Optimisation du Dépôt

Afin de respecter les bonnes pratiques de développement, j'ai mis en place un fichier **.gitignore**.

Ce fichier est crucial car il permet **d'exclure** du serveur les fichiers **lourds ou inutiles**, comme le dossier **de l'environnement virtuel (env/)** ou les dossiers de cache de Python. Cela permet de garder un dépôt GitHub propre, léger et professionnel

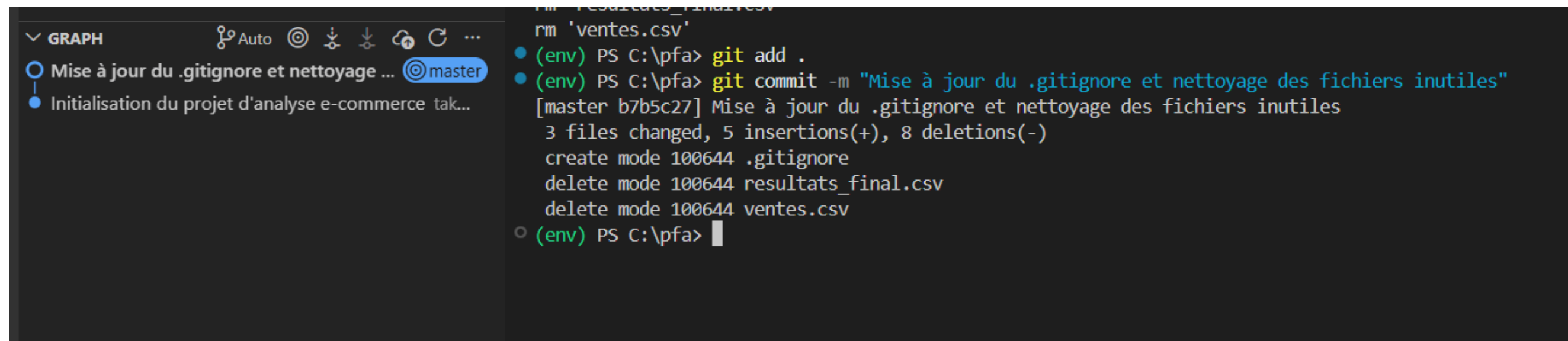


The screenshot shows the Visual Studio Code interface. On the left, the 'SOURCE CONTROL' panel displays a list of files: logo.png, main.py, Readme.md, requirements.txt, and resultats_final.csv. The 'CHANGES' section shows a commit message: 'Message (Ctrl+Enter to commit on master)'. The 'Staged Changes' section shows the same files. The 'Changes' section shows the same files. The 'GRAPH' section shows the commit history. On the right, the editor shows the content of the .gitignore file:

```
# Created by venv; see https://docs.python.org/3/1
env/
__pycache__/
*.csv
.vscode/
```

Below the editor, the 'TERMINAL' panel shows the output of the following commands:

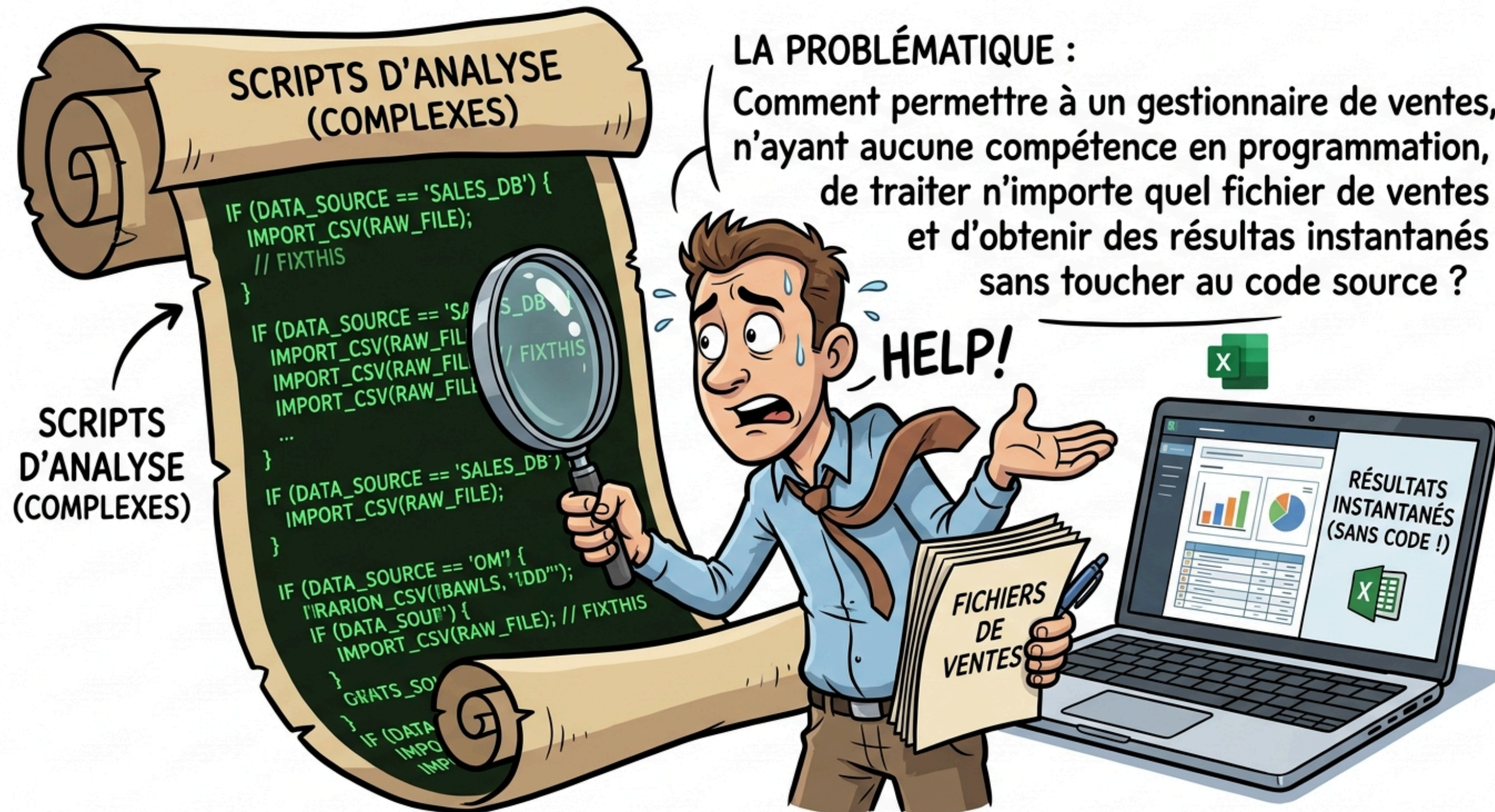
```
cycler 0.12.1
fonttools 4.62.1
kiwisolver 1.5.0
matplotlib 3.10.8
numpy 2.4.4 ...
create mode 100644 Readme.md
create mode 100644 logo.png
create mode 100644 main.py
create mode 100644 requirements.txt
create mode 100644 resultats_final.csv
create mode 100644 ventes.csv
(env) PS C:\pfa> git rm -r --cached .
rm 'Readme.md'
rm 'logo.png'
rm 'main.py'
rm 'requirements.txt'
rm 'resultats_final.csv'
rm 'ventes.csv'
(env) PS C:\pfa>
```



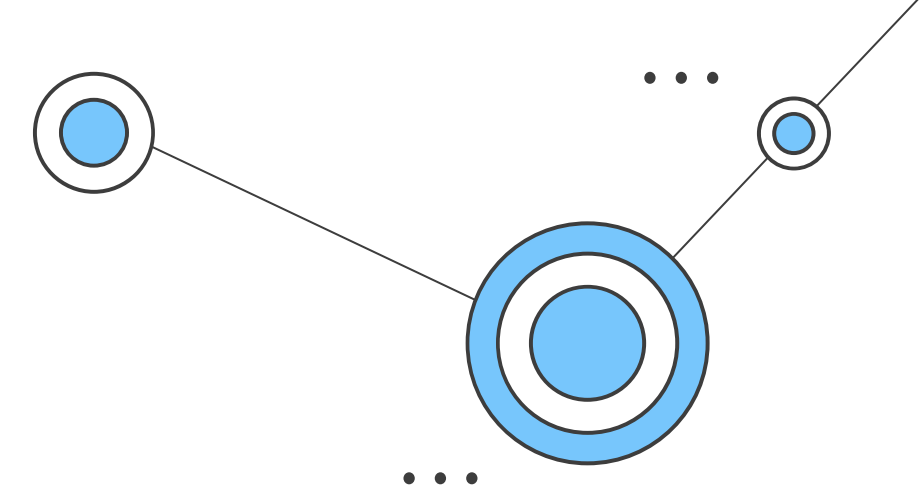
The screenshot shows the Visual Studio Code interface. On the left, the 'GRAPH' panel displays the commit history. The 'Mise à jour du .gitignore et nettoyage ...' commit is selected. The 'Initialisation du projet d'analyse e-commerce tak...' commit is also visible. On the right, the terminal shows the output of the following commands:

```
rm 'resultats_final.csv'
rm 'ventes.csv'
(env) PS C:\pfa> git add .
(env) PS C:\pfa> git commit -m "Mise à jour du .gitignore et nettoyage des fichiers inutiles"
[master b7b5c27] Mise à jour du .gitignore et nettoyage des fichiers inutiles
3 files changed, 5 insertions(+), 8 deletions(-)
create mode 100644 .gitignore
delete mode 100644 resultats_final.csv
delete mode 100644 ventes.csv
(env) PS C:\pfa>
```


Vers une solution métier : Interface interactive avec Streamlit



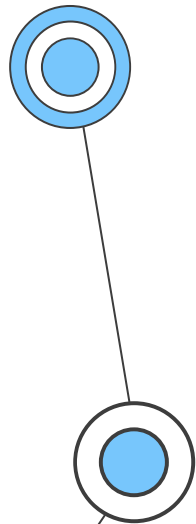
Vers une solution métier : Interface interactive avec Streamlit



La Solution : Notre Interface Interactive (Streamlit)

Pour répondre à ce besoin de simplification, nous avons transformé nos scripts en une application web interactive. Voici les avantages de notre interface :

- **Accessibilité Totale** : Plus besoin d'ouvrir un éditeur de code. L'utilisateur se contente de glisser-déposer son fichier CSV via le bouton d'importation.
- **Traitement Automatisé en Temps Réel** : Dès que le fichier est chargé, l'application exécute les calculs financiers (CA Brut, CA Net avec remises) et met à jour les indicateurs.
- **Tableau de Bord Dynamique (KPI)** : L'interface affiche immédiatement les chiffres clés : le Chiffre d'Affaires total, le produit le plus rentable et la remise moyenne.
- **Visualisation Instantanée** : Un graphique interactif est généré automatiquement pour offrir une lecture visuelle de la performance commerciale.



Vers une solution métier : Interface interactive avec Streamlit

1. Importations et Configuration

- **import streamlit as st** : Importe la bibliothèque principale pour créer l'interface web.
- **import pandas as pd** : Importe l'outil de manipulation de données (DataFrames).
- **import matplotlib.pyplot as plt** : Importe l'outil pour générer les graphiques.
- **st.set_page_config(...)** : Définit le titre de l'onglet du navigateur et utilise toute la largeur de l'écran pour l'affichage.
- **st.title(...)** et **st.markdown("---")** : Affichent le titre principal avec un émoji et une ligne de séparation horizontale.

```
app.py > ...
1  import streamlit as st
2  import pandas as pd
3  import matplotlib.pyplot as plt
4
5  # Configuration de la page
6  st.set_page_config(page_title="Analyseur de Ventes", layout="wide")
7
8  st.title("📊 Analyseur de Ventes E-commerce")
9  st.markdown("---")
```

2. Chargement du Fichier (Étape 1)

- **st.file_uploader(...)** : Crée un bouton sur l'interface permettant à l'utilisateur de ... glisser-déposer son fichier CSV.
- **if uploaded_file is not None:** : Vérifie si un fichier a été chargé avant de lancer les calculs.
- **df = pd.read_csv(uploaded_file)** : Lit le fichier chargé et le transforme en tableau (DataFrame) utilisable par Python.

```
10
11 # Étape 1 : Chargement du fichier
12 st.subheader("📁 Étape 1 : Importer vos données")
13 uploaded_file = st.file_uploader("Choisissez votre fichier CSV", type="csv")
14
15 if uploaded_file is not None:
16     # Lecture des données
17     df = pd.read_csv(uploaded_file)
18
19     # Calcul du CA Brut si non présent
```

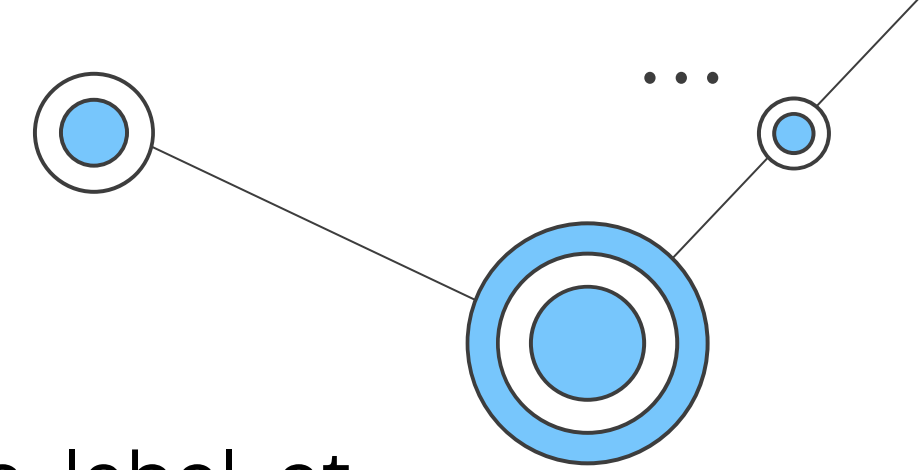

3. Logique de Calcul

- **if 'CA_Brut' not in df.columns:** : Vérifie si la colonne "CA Brut" existe. Sinon, elle est calculée par la formule Prix * Quantite.
- **df['CA_Net'] = ...** : Calcule le chiffre d'affaires net en appliquant la remise (ex: une remise de 20 devient $\$1 - 0.20 = 0.80\$$).
- **total_ca_net = df['CA_Net'].sum()** : Additionne tous les montants de la colonne CA Net.
- **best_id = ...idxmax()**... : Identifie l'ID du produit qui a généré le Chiffre d'Affaires Net le plus élevé.
- **marge_moyenne = df['Remise'].mean()** : Calcule le pourcentage moyen de remise accordé sur l'ensemble des ventes

```
15 if uploaded_file is not None:
16     # Lecture des données
17     df = pd.read_csv(uploaded_file)
18
19     # Calcul du CA Brut si non présent
20     if 'CA_Brut' not in df.columns:
21         df['CA_Brut'] = df['Prix'] * df['Quantite']
22
23     # CORRECTION : On divise la remise par 100 pour le calcul
24     df['CA_Net'] = df['CA_Brut'] * (1 - df['Remise'] / 100)
25
26     # --- CALCUL DES INDICATEURS CLÉS (KPI) ---
27     total_ca_net = df['CA_Net'].sum()
28     # On cherche l'ID du produit qui a le plus grand CA Net
29     best_id = df.loc[df['CA_Net'].idxmax(), 'ID']
30     marge_moyenne = df['Remise'].mean()
31
```

4. Affichage des Indicateurs (Étape 2)

- **st.columns(3)** : Divise l'interface en trois colonnes horizontales.
- **st.metric(...)** : Affiche les résultats (KPI) de manière élégante (Gros chiffre, label, et ... unité €/%).



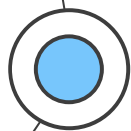
5. Visualisation Graphique (Étape 3)

- **plt.subplots(...)** : Prépare la zone de dessin du graphique.
- **ax.bar(...)** : Crée un graphique en barres utilisant les ID en abscisse et le CA Net en ordonnée. **La couleur #2ecc71 est le vert**
- **st.pyplot(fig)** : Commande spécifique à Streamlit pour intégrer le graphique Matplotlib dans l'interface web.



6. Aperçu des Données et Cas Vide (Étape 4)

- **st.dataframe(...)** : Affiche le tableau complet des résultats. Le `.style.format` permet de n'afficher que 2 chiffres après la virgule pour les colonnes financières.
- **else: st.info(...)** / **st.warning(...)** : Si aucun fichier n'est chargé, l'application affiche des messages d'aide pour guider l'utilisateur.



app.py > ...

```
31
32 # Étape 2 : Affichage des indicateurs
33 st.subheader("💡 Étape 2 : Indicateurs Clés de Performance (KPI)")
34 col1, col2, col3 = st.columns(3)
35
36 with col1:
37     st.metric(label="Chiffre d'Affaires Net Total", value=f"{total_ca_net:,.2f} €")
38
39 with col2:
40     st.metric(label="Produit le plus Rentable (ID)", value=best_id, help="ID du produit avec le plus grand CA Net.")
41
42 with col3:
43     st.metric(label="Remise Moyenne Accordée", value=f"{marge_moyenne:.1f} %")
44
45 st.markdown("---")
46
47 # Étape 3 : Visualisation Graphique
48 st.subheader("📊 Étape 3 : Visualisation des Ventes")
49
50 fig, ax = plt.subplots(figsize=(10, 4))
51 # On change la couleur en vert émeraude comme sur l'image
52 ax.bar(df['ID'].astype(str), df['CA_Net'], color=█ '#2ecc71')
53 ax.set_xlabel("ID Produit")
54 ax.set_ylabel("CA Net (€)")
55 ax.set_title("Chiffre d'Affaires Net par Produit")
56 plt.xticks(rotation=45)
57
58 st.pyplot(fig)
59
60 # Étape 4 : Aperçu des données
61 st.subheader("📄 Étape 4 : Aperçu des données finales")
62 st.dataframe(df.style.format(subset=['Prix', 'CA_Brut', 'CA_Net'], formatter="{:,.2f}"))
63
64 else:
65     st.info("👉 Veuillez importer un fichier CSV pour voir l'analyse.")
66     st.warning("⚠️ Assurez-vous que votre fichier contient les colonnes : ID, Prix, Quantité, Remise")
```

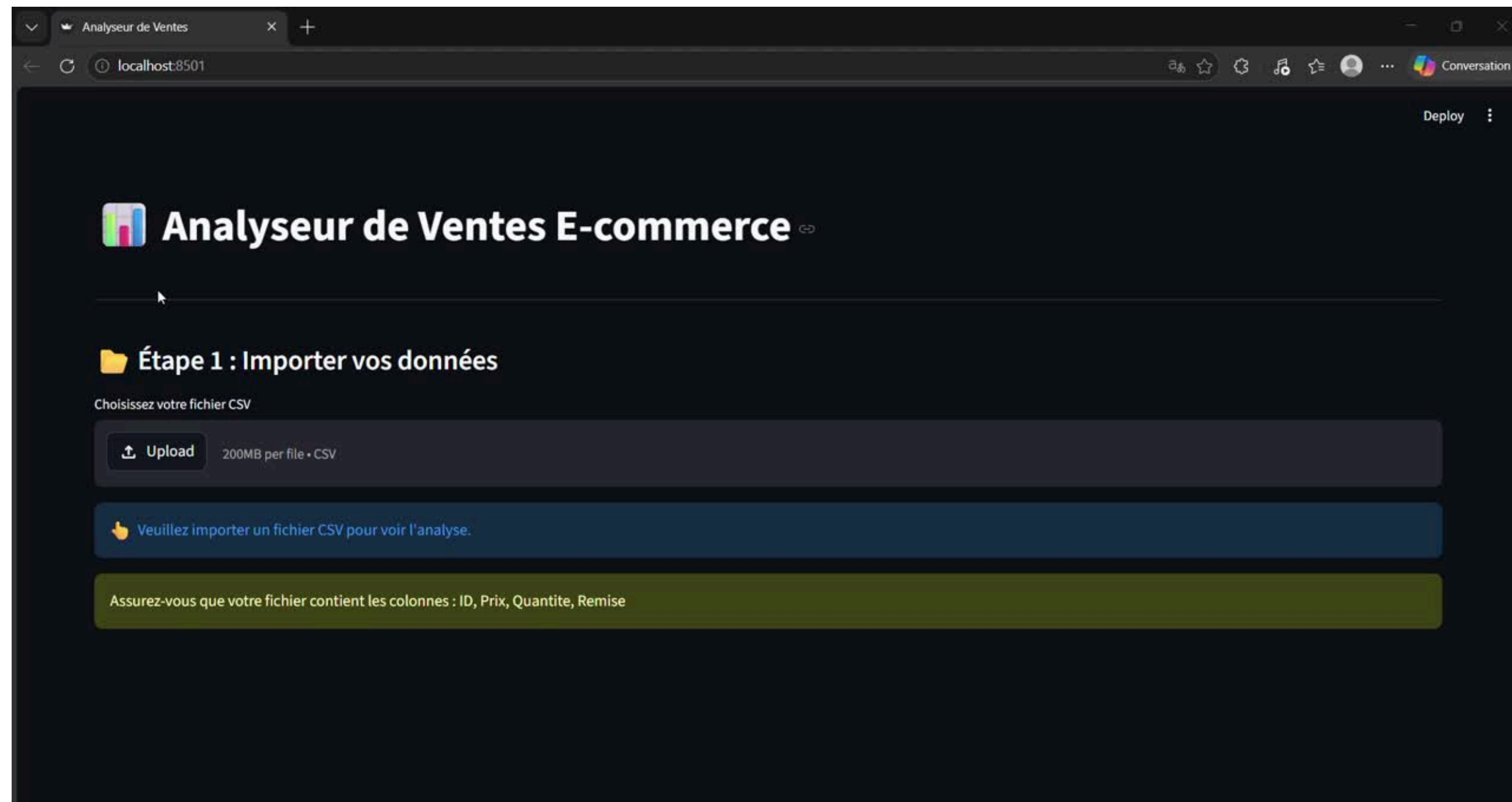
Pour finaliser notre projet, nous passons de la phase de développement à la phase d'exécution :

- **Lancement du serveur** : Via le terminal, nous utilisons la commande **streamlit run app.py**

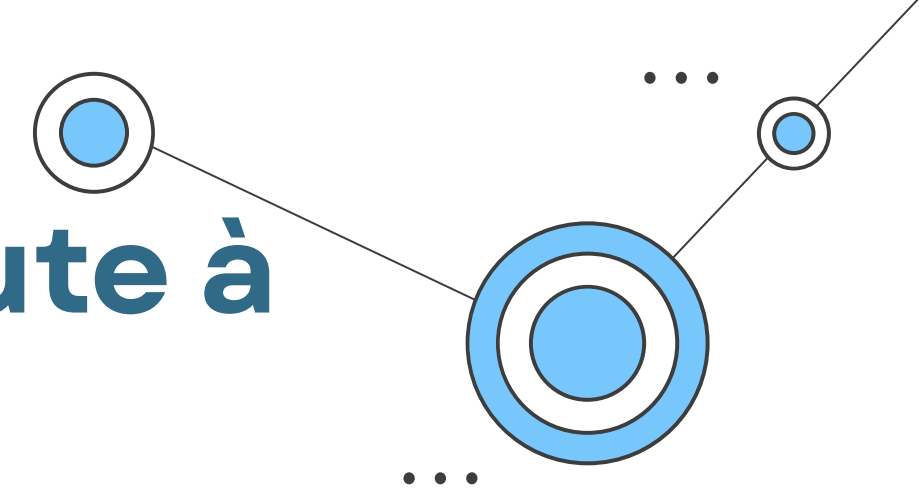
```
(env) PS C:\pfa> streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.19:8501
```

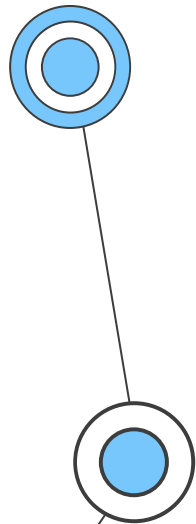


Conclusion Générale : De la Donnée brute à l'Application Interactive



Ce projet a permis de bâtir un flux de travail complet et professionnel en quatre étapes clés :

1. **Infrastructure robuste** : Mise en place d'un environnement virtuel (venv) et d'un suivi rigoureux via Git/GitHub pour garantir la sécurité et la portabilité du code.
2. **Analyse automatisée** : Utilisation de Pandas et Matplotlib dans le script main.py pour automatiser les calculs financiers et la visualisation des résultats.
3. **Test de performance** : Validation de la robustesse du programme sur un jeu de 1 000 données générées par generer_data2.py, simulant un flux réel de vente.
4. **Accessibilité (Streamlit)** : Transformation du script technique en une application web interactive. N'importe quel utilisateur peut désormais importer un fichier CSV et obtenir des indicateurs (KPI) instantanément sans toucher au code.



takwalabidi / Analyse-Ventes-Python

Type / to search

<> Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Analyse-Ventes-Python

Public

Pin

Watch 0

Fork 0

Star 0

main

1 Branch

0 Tags

Go to file

T

Add file

<> Code

takwalabidi

mis a jours du scripts de1000lignes

4a5120a · 2 hours ago

11 Commits

.gitignore	Nettoyage : suppression des images du dépôt et mise à jour ...	3 days ago
Readme.md	mise a jour du README	last week
app.py	ajout de l'interface web interactive avec Streamlit	last week
generer_data.py	Ajout du script de test pour fichier de 500 lignes	2 weeks ago
generer_data2.py	mis a jours du scripts de1000lignes	2 hours ago
logo.png	Initialisation du projet d'analyse e-commerce	2 weeks ago
main.py	Initialisation du projet d'analyse e-commerce	2 weeks ago
main2.py	mis a jours de notre code source	2 hours ago
requirements.txt	mise à jour des dépendances	last week

About

Projet académique de traitement de flux de données CSV. Mise en œuvre d'un environnement virtuel isolé, gestion de versions avec Git et analyse statistique de fichiers de ventes volumineux

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

Merci pour votre attention

Nos E-mail:

takwalabidi.5@gmail.com

Hammok05@gmail.com

lien github:<https://github.com/takwalabidi/Analyse-Ventes-Python>